

CHAPTER- 1

INTRODUCTION

The ever-evolving field of software development demands developers who are proficient in both frontend and backend technologies. Full-stack development, in particular, requires an intricate understanding of how various layers of a web application interact to provide seamless, efficient, and user-friendly experiences. To bridge the gap between academic knowledge and real-world application, I undertook an internship in Java Full Stack Development at Pentagon Space, a technology firm renowned for its innovative solutions and cutting-edge software products.

During this internship, I was not only introduced to cutting-edge technologies but also immersed in the practical aspects of building and maintaining real-world applications. Pentagon Space operates in a fast-paced, collaborative environment where software solutions are developed for a wide range of clients. This exposure provided me with a unique opportunity to work on live projects, gaining first-hand experience in the entire software development lifecycle (SDLC). I had the privilege of contributing to a variety of tasks, from designing intuitive user interfaces to building robust backend systems, and gained significant insight into the complex architecture that supports modern web applications.

At Pentagon space, my role as a Java Full Stack Developer Intern required me to participate in both frontend and backend development activities, giving me a holistic view of how different technologies interact to form a complete application. I worked with React.js for frontend development, utilizing its component-based architecture to create responsive, dynamic, and user-friendly web pages. In parallel, I used Java and Spring Boot for backend development, building RESTful APIs that facilitated communication between the user interface and the server. By combining these technologies, I was able to contribute to the development of web applications that were both highly functional and visually appealing.

The backend development experience was especially insightful. I had the opportunity to work with Hibernate, which simplified the process of interacting with relational databases, and MySQL, where I learned how to design efficient database schemas and write optimized queries. I also gained experience in managing database connections and ensuring data integrity while optimizing the performance of the system.

COMPANY PROFILE

Pentagon Space Pvt. Ltd. is a dynamic and rapidly growing Indian IT services and technology solutions provider, headquartered in Bangalore, Karnataka. Established in 2020, the company delivers a comprehensive range of cutting-edge digital solutions, software development services, and consulting offerings tailored to meet modern business needs.

Leveraging a strong foundation in innovation and a highly skilled team of professionals, Pentagon Space partners with clients across various industries to help them transform digitally, optimize operations, and drive sustainable growth. The company's expertise spans full-stack development, enterprise solutions, software testing, and artificial intelligence.

Vision: To become a global leader in digital transformation by offering innovative, reliable, and scalable IT solutions that empower clients to excel in the digital era.

Mission: To deliver high-quality, technology-driven services by leveraging deep domain knowledge and a commitment to excellence, thereby building long-term value and relationships with our clients.

Core Services

- **Custom Software Development:**
Tailored software applications designed to solve specific business problems using modern frameworks and cloud technologies.
- **Web and Mobile App Development:**
Full-cycle development of responsive, secure, and user-centric web and mobile apps across platforms.
- **Enterprise Resource Planning (ERP) Solutions:**
Streamlined enterprise-wide systems that enhance productivity, data integration, and decision-making.
- **Software Testing and QA:**
Robust manual and automation testing services ensuring product quality, functionality, and reliability.
- **Digital Transformation Consulting:**
Strategic consulting to help businesses modernize legacy systems and adopt digital tools.

Technology Stack

- Backend: Java, Python, Node.js
- Frontend: React, Angular, HTML/CSS
- Databases: MySQL, MongoDB, PostgreSQL
- AI/ML: TensorFlow, scikit-learn, OpenCV
- Cloud: AWS, Azure
- Testing: Selenium, JUnit, TestNG

Key Clients & Partnerships

- Collaborations with enterprises and startups across finance, retail, manufacturing, education, and defence sectors.
- Strategic technology partner for numerous product companies and service-based firms.
- Trusted vendor for talent augmentation, product design, and IT modernization projects.

Why Pentagon Space?

- Agile and scalable IT solutions
- Dedicated team with cross-domain expertise
- Client-centric engagement models
- Focus on innovation and continuous improvement
- Proven track record in delivering real-world impact through technology

CHAPTER- 2

OBJECTIVES OF INTERNSHIP

The primary goal of this internship was to develop a comprehensive understanding of the end-to-end software development process, particularly in the context of Java Full Stack technologies. More specifically, the objectives were:

- **To gain proficiency in both frontend and backend technologies:** The role involved working with Java-based backend technologies, including Spring Boot for API development, Hibernate for ORM (Object-Relational Mapping), and MySQL for database management. On the frontend, I worked with React.js, HTML5, CSS3, and JavaScript to design and implement dynamic and responsive user interfaces.
- **To understand the software development lifecycle (SDLC):** I participated in all stages of development, from initial requirements gathering and system design to implementation, testing, deployment, and post-deployment support.
- **To integrate backend services with frontend applications:** I learned how to establish communication between the frontend and backend using RESTful APIs, and how to ensure seamless data flow and system performance across multiple components.
- **To adopt modern development tools and practices:** The internship exposed me to tools such as **Git** for version control, **Postman** for API testing, and Agile methodologies for project management. These tools and practices are integral to modern software development teams and ensured a professional and collaborative work environment.

CHAPTER- 3

INTERNSHIP ACTIVITIES

During the course of my internship as a **Java Full Stack Developer**, I was exposed to a wide range of practical tasks and collaborative activities that enhanced both my technical and professional skillset. The internship followed a structured development life cycle model and included real-world assignments using both frontend and backend technologies. Below is a breakdown of the core activities:

3.1 Development Environment Setup

Installed and configured essential tools:

- **Eclipse IDE** for Java and Spring Boot backend development.
- **Visual Studio Code** for React.js frontend development.
- **Postman** for testing and validating RESTful APIs.
- **MySQL Workbench** for managing databases and executing SQL queries.

Cloned project repositories from GitHub using Git and set up local branches for feature development.

Configured dependency managers:

- **Maven** for managing Java backend dependencies.
- **Node Package Manager (npm)** for installing and managing frontend packages.

Resolved initial setup issues such as:

- Missing or outdated dependencies.
- Compilation and runtime errors.
- Inconsistent version conflicts between modules.

Studied the project architecture to understand the structure of the application, including:

- Frontend and backend folder hierarchy.
- Service layer, controller layer, and data access layer.
- Component-based frontend structure in React.

Configured environment variables and properties files, including:

- application.properties for Spring Boot backend (e.g., server port, database URL).
- .env files for frontend APIs and keys.

Validated the development setup by:

- Running the backend Spring Boot server locally.
- Launching the frontend React app on the browser.
- Testing initial API endpoints using Postman.

Established a smooth development workflow by integrating tools with GitHub for version control, ensuring CI/CD compatibility where required.

3.2 Frontend Development

During the Java Full Stack Internship, a major focus was placed on mastering frontend development technologies and practices to create responsive, interactive, and scalable web applications. Leveraging modern frameworks and libraries such as React.js, combined with core web technologies like HTML5, CSS3, and JavaScript, I gained hands-on experience in building user-friendly interfaces. I worked extensively with state management tools, asynchronous API integrations, and UI design systems to ensure both functionality and aesthetic consistency in the applications developed.



Fig: 3.2 Frontend Development

3.2.1 Built Responsive UI Components

- Utilized **React.js** to develop reusable and modular components.
- Applied **HTML5 and CSS3** to create semantic, structured, and well-styled UI elements.
- Used **JavaScript** to add interactivity and dynamic behaviour to the web pages.
- Ensured responsiveness across devices using **CSS Flexbox, Grid, and media queries**.

3.2.2 Implemented State Management

- Managed local states using **React Hooks** like useState, useEffect, etc.
- Applied **Redux** for centralized and predictable global state management.
- Created actions and reducers to handle user interactions and update the UI accordingly.
- Organized Redux store to manage data flow and improve application scalability.

3.2.3 Integrated APIs and Handled Asynchronous Data

- Used **Axios** and the **Fetch API** to connect frontend components to backend services.
- Implemented **CRUD operations** for sending and receiving data from RESTful APIs.
- Managed asynchronous operations with async/await, ensuring smooth user experiences.
- Handled loading states, API errors, and empty data gracefully with proper UI feedback.

3.2.4 Ensured UI Consistency with Component Libraries

- Employed **Material UI** to build sleek, material-design-based components.
- Used **Bootstrap** for responsive design layouts and utility classes.
- Customized themes and applied consistent styling across pages.
- Maintained visual consistency and improved accessibility using predefined design patterns.

3.3 Backend Development



Fig:3.3 Backend Development

- 1. Developed RESTful APIs:** Created APIs using Spring Boot for frontend-backend communication, handling user registration, login, and data retrieval.
- 2. Implemented Business Logic:** Wrote service classes to manage core logic, ensuring proper data validation and processing before database interaction.
- 3. Connected to Databases:** Integrated MySQL for data storage and used JPA/Hibernate for CRUD operations and entity relationships.
- 4. Handled Security and Authentication:** Secured APIs using Spring Security and JWT for authentication and role-based access control.
- 5. Performed Data Validation and Error Handling:** Used Java Bean Validation for input validation and implemented custom error handling with Spring's `@ControllerAdvice`.
- 6. API Testing and Documentation:** Tested APIs with Postman, used Swagger/OpenAPI for documentation, and wrote unit/integration tests with JUnit and Mockito.

3.4 Database Design and Querying

Database design involves creating a structured database system that is optimized for data storage, retrieval, and manipulation. The goal is to ensure the database is scalable, reliable, and maintains data integrity. Key components of database design include:

- Entity Relationship Diagram (ERD):** The first step in database design is creating an ERD, which outlines the relationships between various entities (tables). It helps in visualizing how different pieces of data relate to each other, which is crucial for ensuring data consistency.
- Normalization:** Database normalization is the process of organizing the data in the database to minimize redundancy and improve data integrity. This involves dividing large tables into smaller ones and defining relationships between them using foreign keys.



Fig: 3.4 Database

- Database Management System (DBMS) is the core system that manages data storage, retrieval, and updates, ensuring integrity and consistency.
- Query Language & Processor handle the execution of user queries, allowing interaction with data through structured commands (e.g., SQL).
- Storage Engine manages the physical storage of data, working with the database files to retrieve and write data efficiently.
- Access Control & Policies secure the system by managing user roles, permissions, and data protection rules.
- Monitoring & Reporting provide insights into system performance and generate analytical reports, supported by logs and performance metrics.

CHAPTER- 4

PROJECT INFORMATION

(PetHub)

In today's digital era, the way individuals interact with everyday services has undergone a significant transformation. From online shopping and education to healthcare and transportation, nearly every sector has adopted technology to provide users with enhanced convenience and efficiency. The pet care industry, however, remains one of the relatively under-digitized domains, especially when it comes to combining pet management, healthcare, and product services under a unified system. Recognizing this gap, PetHub was conceptualized and developed as a comprehensive web-based solution aimed at modernizing the experience of pet owners, simplifying their responsibilities, and improving pet well-being through a smart, centralized system.

PetHub is designed to be an interactive and user-friendly platform that allows users to register on the system, manage the details of their pets, book veterinary appointments, and purchase necessary pet-related products such as food, accessories, and snacks. Unlike traditional pet care models where users might need to rely on different service providers for each need—vets, pet stores, record keeping PetHub aims to streamline all these services in one integrated digital environment. This system caters specifically to owners of commonly domesticated pets like dogs, cats, birds, and fish, enabling them to access essential services without leaving their homes.

One of the key innovations of PetHub lies in its Pet Management Module, which allows users to enter and update their pets' details such as name, type, breed, age, gender, medical history, vaccination dates, and more. This module is the core around which other functionalities operate. For instance, when booking a vet appointment, the user can select from their list of registered pets, providing the doctor with accurate context and health background. This not only saves time but also enables personalized and better-informed veterinary care.

Another critical module is the Doctor Appointment System, where users can select the type of pet, choose a time slot, and schedule a consultation with an available veterinarian. This ensures that pet owners can proactively manage their pets' health and wellness without the traditional hassle of physically visiting a clinic for each appointment or inquiry. The system is designed to notify users of upcoming appointments and also allows rescheduling or cancellations when needed.

In addition to health care services, PetHub also includes a Pet Product Shopping Module where users can browse and purchase a variety of items tailored to their pets' needs. These items include premium pet foods, toys, grooming tools, accessories, and treats. Each product is categorized based on pet type, making it easy for users to find what they need. Orders placed by users are tracked and managed by the admin, who oversees product delivery to the provided addresses. This level of integration ensures a seamless experience for pet owners, who no longer need to use multiple apps or websites to meet their pets' daily needs.

From an administrative perspective, PetHub provides a secure admin panel where system administrators can log in and monitor all system activities. The Admin Module is responsible for managing doctor appointments, confirming or rejecting bookings, updating order delivery status, managing product inventory, and reviewing overall system performance. Admins can also generate reports to evaluate usage patterns, user engagement, and transaction details, which is helpful for improving system services over time.

The system is built using a robust Java Full Stack approach, including Spring Boot for the backend, HTML/CSS and optionally React.js for the frontend, and MySQL for data storage. RESTful APIs are implemented to ensure smooth communication between the frontend and backend. All user and admin interactions go through a secure authentication process to maintain data privacy and prevent unauthorized access. Optional use of Spring Security can further enhance this aspect. Tools like Postman are used for API testing, and GitHub is used for version control and team collaboration.

One of the motivations behind PetHub is the increasing number of households that own pets and the corresponding increase in demand for quality pet care services. As pet ownership continues to grow globally, the need for efficient, digital-first solutions becomes more apparent. Pet owners today are tech-savvy and expect the same level of service quality and digital access that they get in other domains like food delivery, e-commerce, and healthcare. PetHub directly addresses these expectations by creating a one-stop solution that not only meets their basic requirements but also offers a personalized, intuitive, and dependable service ecosystem.

4.1 OBJECTIVES OF PROJECT

1. To create a centralized platform for pet owners to manage pet details, book veterinary appointments, and shop for pet products.
2. To enable users to add, update, and manage information for multiple pets, including their health records, breed, age, and medical history.
3. To implement an online appointment booking system that allows users to schedule and track veterinary consultations for their pets.
4. To provide an integrated online store where users can browse and purchase pet food, accessories, snacks, and other essentials.
5. To allow admins to manage appointments, monitor order deliveries, and maintain inventory of products efficiently from a secure backend.
6. To ensure a user-friendly, responsive interface that is accessible on both desktop and mobile devices, enhancing the overall user experience.

4.2 METHODOLOGY

4.2.1 User Registration & Authentication:

This flowchart clearly outlines the user login and registration process in the PetHub application. It begins with the start point, where the user initiates interaction by opening the PetHub platform. Upon launching the application, the system immediately checks whether the user is already logged in. This decision point is marked with a diamond-shaped decision node labeled “Is User Logged In?”. This node forms the core of the logic that separates returning users from new users and determines the appropriate next step.

If the answer is “No”, meaning the user is not logged in, the flow proceeds to the Register/Signup step. Here, the user is prompted to create a new account. In the following step, the user is asked to enter necessary details such as their name, email, password, and type of pet (e.g., dog, cat, bird, or fish). These details are essential for uniquely identifying the user and personalizing their experience on the platform.

Once the user fills in the required information, the next step involves storing this data securely in the database. This storage process ensures that user credentials and pet information are saved for future logins and profile access. After successful registration, the user is then directed to log in with the newly created credentials, just like any other existing user.

Meanwhile, if the decision at the “Is User Logged In?” point is “Yes”, indicating the user has previously registered and their session is recognized or valid login credentials are provided, the flow bypasses the registration steps. The user directly proceeds to the login screen, where they input their email and password.

Once credentials are submitted and verified, the system grants access to the user dashboard. This dashboard is the central interface where users can manage their pet’s health records, book appointments, shop for products, and receive updates or notifications. It acts as the command center for all PetHub features tailored to the user’s account.

The flowchart demonstrates a well-structured and user-friendly approach to onboarding and accessing the application. It clearly separates new user actions from those of returning users, which helps improve usability and reduces confusion. The database interaction ensures persistent data management, which is crucial for delivering personalized services.

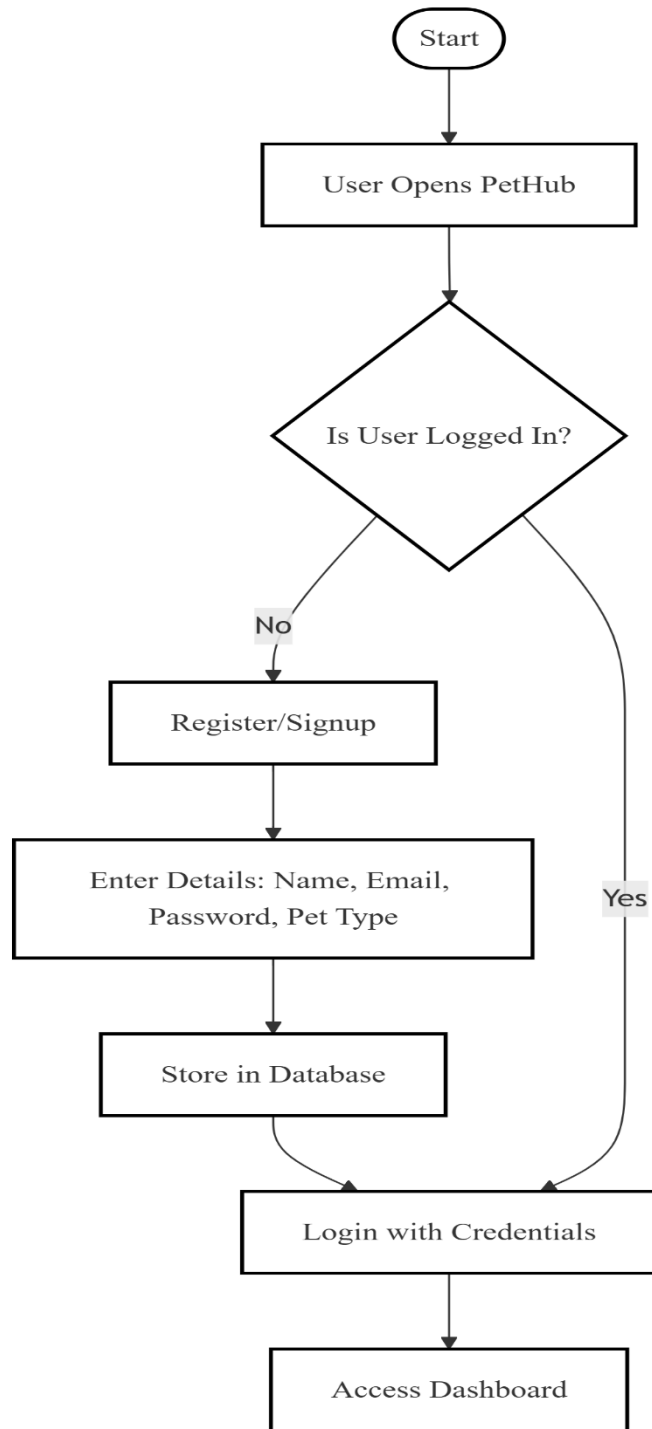


Fig 4.2.1: User Registration & Authentication for PetHub

4.2.2 Vet Appointment Booking:

The PetHub veterinary appointment system provides a streamlined digital solution for pet owners to schedule care for their animals. Beginning with user authentication, the platform verifies credentials before granting access to booking features. When selecting "Book Appointment," users first specify their pet type (dog, cat, bird, or fish), which tailors the interface and filters appropriate veterinarians. The system then displays available doctors based on specialization, location, and user ratings, pulling real-time availability data to prevent scheduling conflicts. Users browse time slots color-coded by urgency (routine, follow-up, emergency) with duration estimates based on pet size and visit type.

After selecting a preferred slot, the system confirms the doctor's immediate availability through a quick backend check before proceeding. Payment integration activates for services requiring deposits, with transparent pricing displayed upfront. Upon confirmation, the appointment is instantly logged in MongoDB with a unique ID, triggering multiple automated processes: the vet receives an SMS/email notification with pet details, the user's app calendar updates, and the admin dashboard reflects the new booking. The database updates include cascading changes to the vet's availability profile and clinic resource allocation. For clinics, the system auto-assists exam room scheduling and prepares digital intake forms based on the pet's medical history. Two-way sync with practice management software ensures clinics see the same updated schedules. The admin interface provides tools for rescheduling, with conflict detection that suggests alternative slots.

Automated reminders are scheduled - 24 hours before the appointment via the user's preferred channel (push notification, email, or SMS), with a follow-up 1-hour prior. The vet's dashboard displays the day's appointments with pet profiles, medical alerts, and owner notes. Integration with telehealth enables virtual check-in options when applicable. Post-visit, the system prompts users to rate their experience and schedule follow-ups, while updating the pet's digital health records. For cancellations, a grace period allows other users to claim newly available slots through real-time alerts. The entire flow is optimized for mobile use, with offline capability that syncs data when connectivity returns. Security measures encrypt all health data transmissions, while audit logs track every booking modification. Performance monitoring ensures the calendar loads in under 2 seconds even during peak demand.

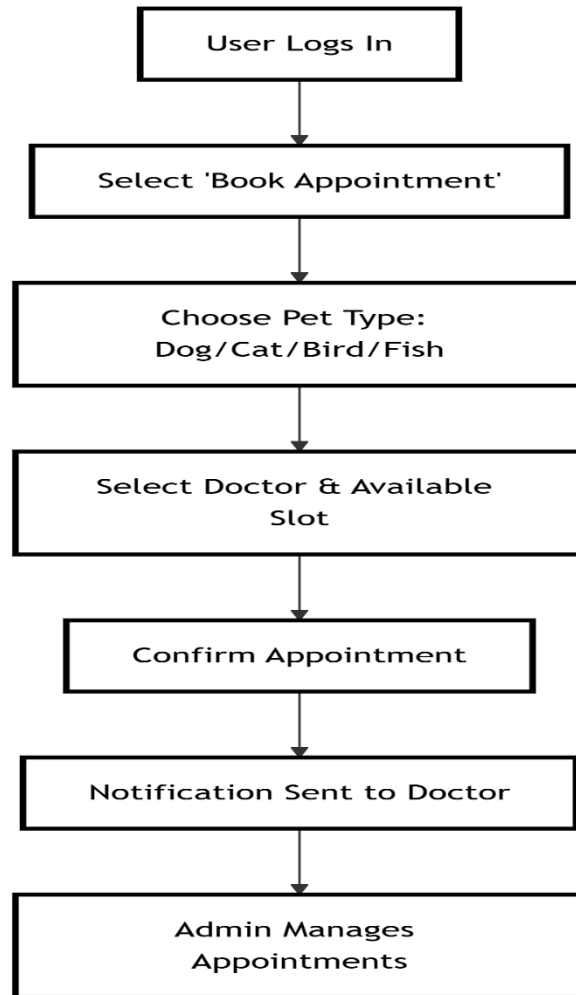


Fig 4.2.2: Vet Appointment Booking for PetHub

4.2.3 E-Commerce Order Flow:

The PetHub e-commerce flowchart illustrates a seamless and structured workflow from user login to order delivery. The process begins with user authentication, ensuring that only registered users can access the shopping features. Once logged in, users can browse a categorized catalog of pet products such as food, accessories, and care items tailored to specific pet types. After selecting products, users add them to a virtual cart and proceed to the checkout phase. During checkout, they enter a valid shipping address, which can be verified using integrated mapping APIs for accuracy. Users then select a preferred payment method—such as credit cards, digital wallets, or cash on delivery with all financial transactions secured through encryption. Upon confirming the order, the system stores it in the database, triggering backend operations.

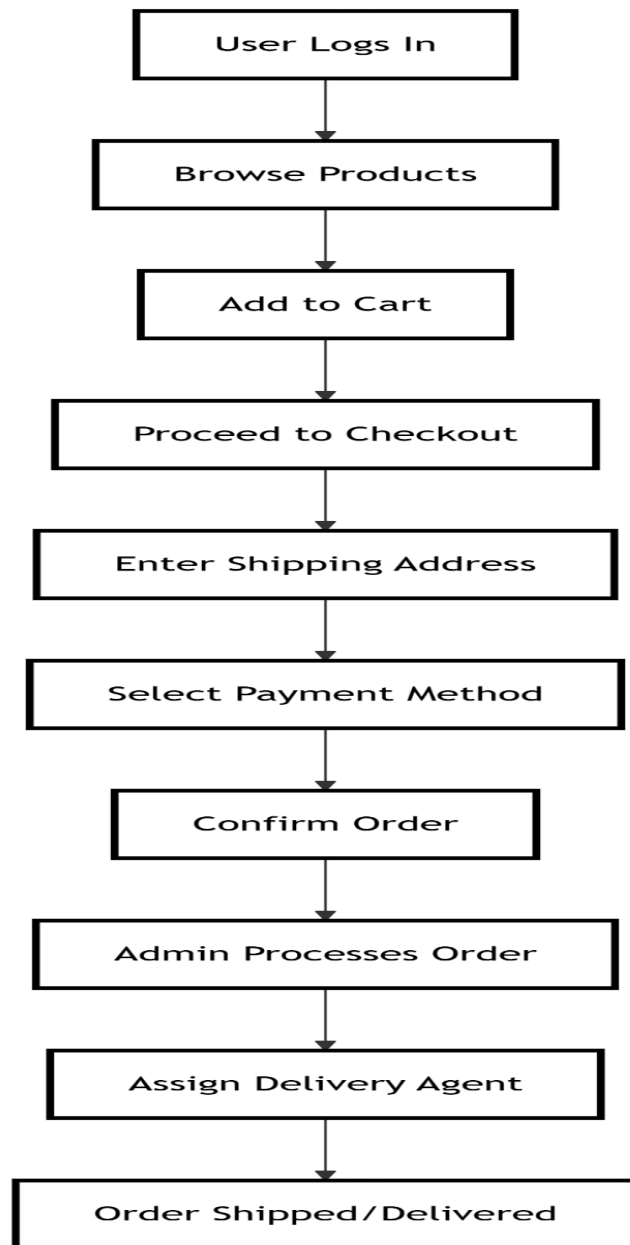


Fig 4.2.3: E-Commerce Order Flow for PetHub

4.2.4 Admin Management Flow:

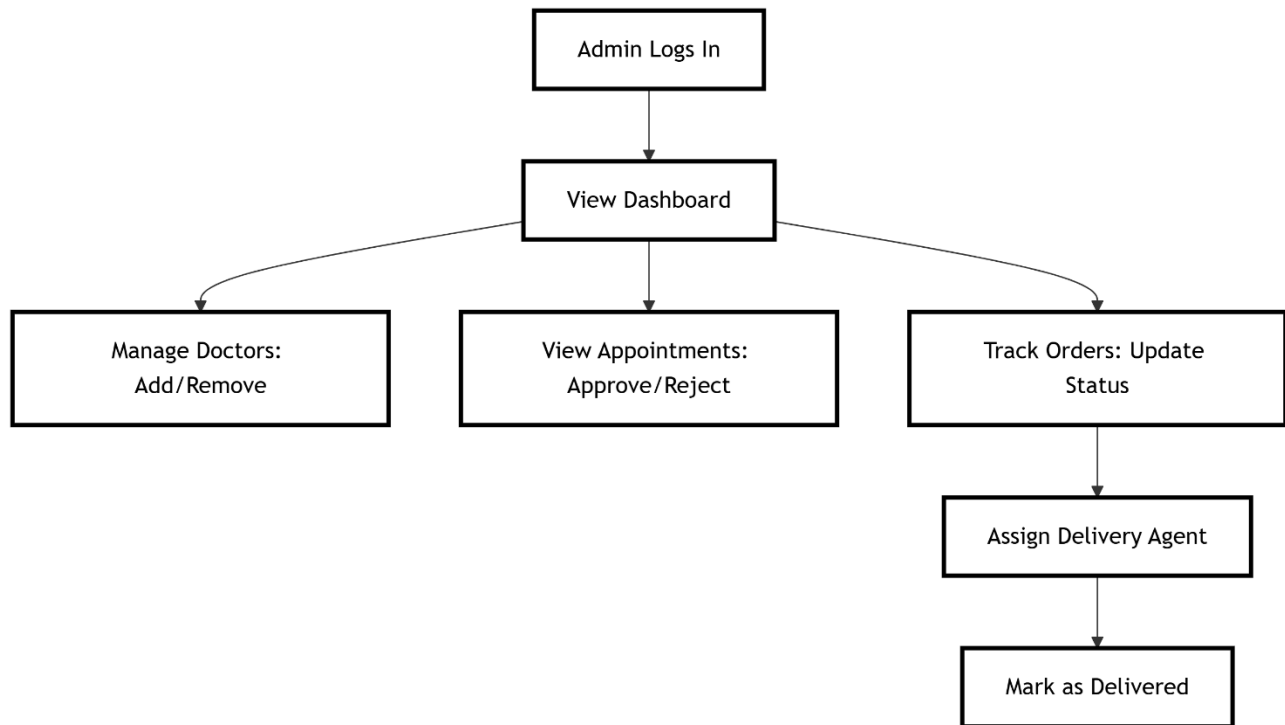


Fig 4.2.4: Admin Management Flow for PetHub

The admin workflow flowchart for PetHub shows how administrators manage the platform efficiently and securely. It starts with a secure login that uses multi-factor authentication and role-based access. Once logged in, the admin sees a dashboard with real-time data like upcoming appointments, order statuses, and doctor availability. The first task is managing doctors—adding or removing them and updating their schedules, which automatically updates the public booking system. Admins can then view and approve or reject appointments, using filters and tools to avoid double bookings or errors. For e-commerce, they track orders through a clear system showing each step from payment to delivery. They can assign delivery agents based on location and workload, and updates go out instantly to both agents and customers. Once a delivery is confirmed, the system marks it as complete, sends a receipt, and may ask for feedback. The system includes helpful features like alerts for low inventory, color-coded priorities, and secure logs of admin activity. It supports both desktop and mobile use and is built to grow with future needs.

4.2.5 Delivery Tracking:

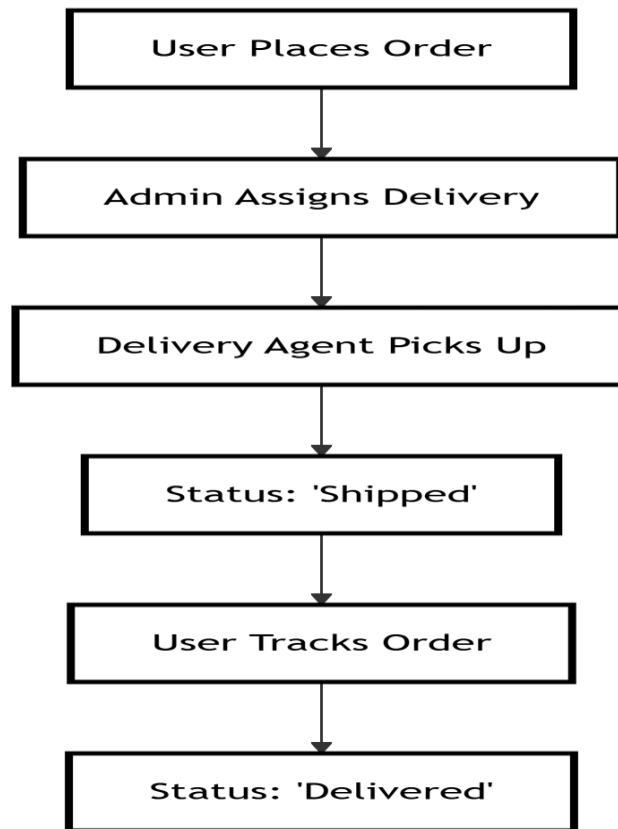


Fig 4.2.5: Delivery Tracking for PetHub

The order fulfillment process at PetHub starts when a customer places an order, and the system instantly creates a detailed order record while updating stock levels. Next, the admin checks the order details, confirms payment, and assigns a delivery agent using smart algorithms that consider location and availability. The assigned agent picks up the package from the warehouse or vendor, scans it to confirm collection, and updates the order status to "Shipped." Customers are then notified and can track their order in real time using an interactive map. The system also shares delivery estimates and updates if there are any delays. Once the package is delivered, the agent confirms it through a signature, photo, or GPS verification. The system marks the order as "Delivered" and sends a digital receipt while also offering feedback options and loyalty rewards. All data is kept secure with encryption and the platform follows strict privacy rules. IoT devices help monitor conditions for sensitive items, and predictive tools avoid delivery delays. Support features like messaging and issue resolution make the process smooth and user-friendly.

4.3 ER DIAGRAM

This Entity-Relationship Diagram (ERD) illustrates the database structure of the PetHub application by defining the main entities, their attributes, and relationships. At its core is the User entity, which includes id, name, email, password, and address. Each user can own multiple Pets, indicating a one-to-many relationship between users and pets. The Pet entity holds details about animals registered by users.

The Admin entity is responsible for operational tasks and is linked to Appointments, which they help schedule for pets. The Appointment entity includes attributes like id, pet_id, and date, allowing proper tracking of scheduled medical services for pets.

Products are associated with Pets since pet owners purchase items like food or accessories for them. The Order entity records purchase details and includes id, category, price, and user_id. The relationship between Product and Order, labeled as "buys", indicates users place orders for pet products.

The user_id in the Order table links back to the User, keeping track of who bought what. This enables efficient transaction management and personalized user services. The connection between Appointment, Pet, and Admin reflects a well-coordinated veterinary booking system.

By clearly separating entities such as Appointment and Order, the system ensures better organization and management of services and purchases. Each foreign key, like pet_id and user_id, maintains referential integrity and ensures that data is consistently linked across tables.

This model supports core PetHub functionalities like user registration, pet management, appointment scheduling, and order processing. It enables features such as real-time bookings, product tracking, and user dashboards. Overall, the ERD captures a comprehensive view of how users, pets, admins, products, and services interact within the platform.

Its clean design ensures scalability and future expansion. Additional features like payment gateways, notification systems, and analytics tools can be added without disrupting existing structures. The ERD offers a reliable foundation for PetHub's digital pet care ecosystem.

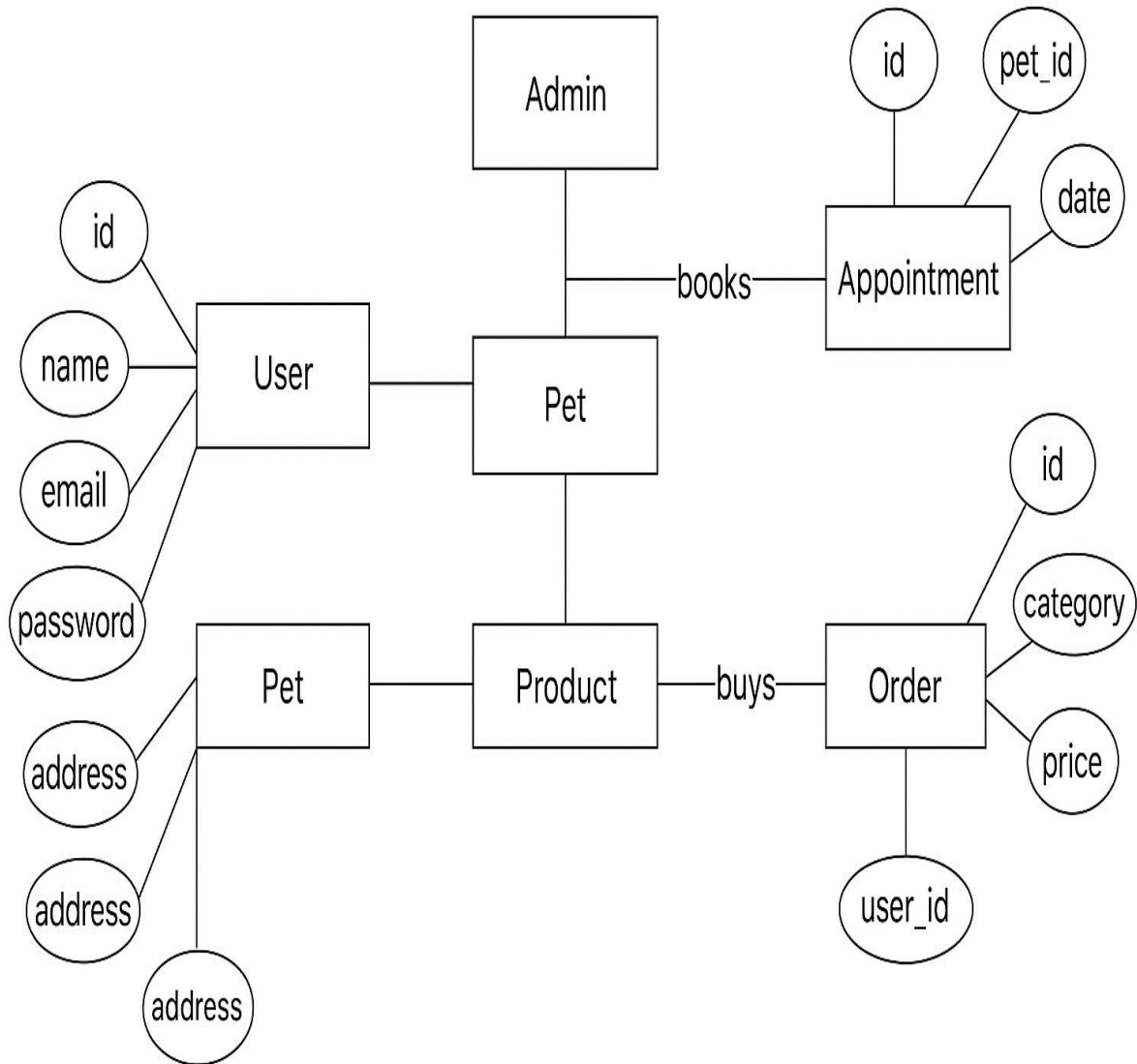


Fig 4.3s: ER Daigram for PetHub

4.4 SYSTEM DESIGN

This diagram represents the three-tier architecture of the Pet Management System, breaking it down into the Presentation Layer, Application Layer, and Data Layer. At the top, the Presentation Layer is built using React, which provides a user-friendly interface for both Pet Owners and Admins. Users interact with the system through this layer, performing actions like booking appointments or managing pets. When users take any action, API requests are sent from the React frontend to the backend.

The Application Layer is powered by Spring Boot, which acts as the core logic engine of the application. It processes the requests, applies business rules, and communicates with the database. This layer handles functionalities like validating appointments, processing orders, and assigning deliveries. After processing, it sends API responses back to the React frontend to update the user interface.

The Data Layer uses MySQL, a relational database that stores all the data related to users, pets, appointments, products, and orders. Spring Boot communicates with MySQL using repositories and queries to retrieve or update data as needed. This layered design ensures separation of concerns, making the application more modular, scalable, and easier to maintain. It also promotes secure and efficient data flow, where the frontend is independent from the backend logic and data storage. Overall, this architecture enables a smooth and structured workflow between users, services, and data.

This diagram illustrates a three-tier architecture for a Pet Management System, clearly separating concerns across Presentation, Application, and Data layers. At the top, the Presentation Layer is powered by React, a popular JavaScript library used to build interactive user interfaces. This layer serves two types of users: Pet Owners, who can register pets, book appointments, and order products, and Admins, who manage appointments, products, and deliveries. All interactions on the front end, such as clicking buttons or filling out forms, trigger API requests that are sent to the backend.

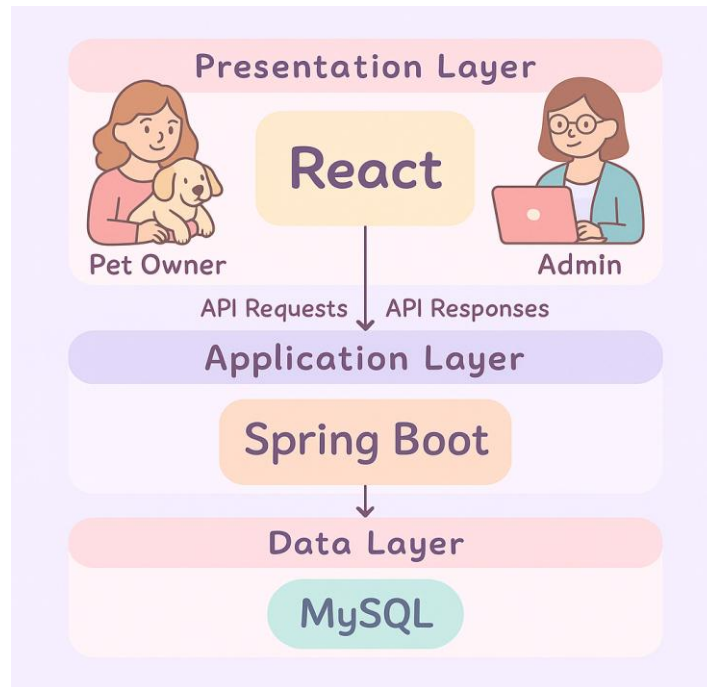


Fig 4.4: System Design for PetHub

These requests are handled in the Application Layer, which is built using Spring Boot, a powerful Java-based backend framework. This layer acts as the brain of the system, managing business logic, processing input, validating data, and responding to frontend queries. It ensures that each request is correctly handled whether it's saving pet data, scheduling an appointment, or generating delivery updates. The Application Layer sends back API responses that update the user interface with the latest data, making the system dynamic and responsive.

Below that is the Data Layer, which uses MySQL, a relational database that stores all core information securely. This includes user credentials, pet profiles, appointment details, product inventories, orders, and admin records. Spring Boot connects to the database through data access layers like JPA or JDBC, running SQL queries to fetch, update, delete, or insert data. The structure ensures data integrity, consistency, and security.

This layered approach offers several benefits: it enhances scalability, simplifies maintenance, and improves security by isolating each layer. For example, changes in the UI (React) don't directly affect backend logic (Spring Boot) or the database structure (MySQL). This modularity makes it easier to add new features like notifications, payment modules, or veterinary chat support without disturbing the entire system. Overall, this architecture ensures a seamless user experience while maintaining robust backend operations.

4.5 SEQUENCE DAIGRAM

The given sequence diagram illustrates the interaction between various components involved in a Pet Management System, such as users, web application UI, controllers, services, and the database. The diagram outlines a sequential flow of actions that occur from user login to order placement and appointment scheduling, helping visualize how the backend processes are managed and how services communicate with each other.

At the beginning, the user starts by logging into the system using an email and password via the Web Application UI. The UI component sends these credentials to the backend to validate them. Upon successful validation, the user is authenticated, granting them access to perform further actions such as adding pets, placing orders, or viewing appointments.

Once logged in, the user can add a new pet. This action is initiated from the UI, which sends a POST /add Pet request to the Appointment Controller. This controller then forwards the savePet(petData) request to the Appointment Service, which interacts with the database to insert the pet's information. The pet information typically includes name, breed, age, and the owner's details. Once the pet is successfully added, the system may notify the admin or keep a record of this action for future appointments or services.

The given sequence diagram illustrates the interaction between various components involved in a Pet Management System, such as users, web application UI, controllers, services, and the database. The diagram outlines a sequential flow of actions that occur from user login to order placement and appointment scheduling, helping visualize how the backend processes are managed and how services communicate with each other.

At the beginning, the user starts by logging into the system using an email and password via the Web Application UI. The UI component sends these credentials to the backend to validate them. Upon successful validation, the user is authenticated, granting them access to perform further actions such as adding pets, placing orders, or viewing appointments.

Once logged in, the user can add a new pet. This action is initiated from the UI, which sends a POST/add Pet request to the Appointment Controller. This controller then forwards the savePet(petData) request to the Appointment Service, which interacts with the database to insert the pet's information.

The pet information typically includes name, breed, age, and the owner's details. Once the pet is successfully added, the system may notify the admin or keep a record of this action for future appointments or services.

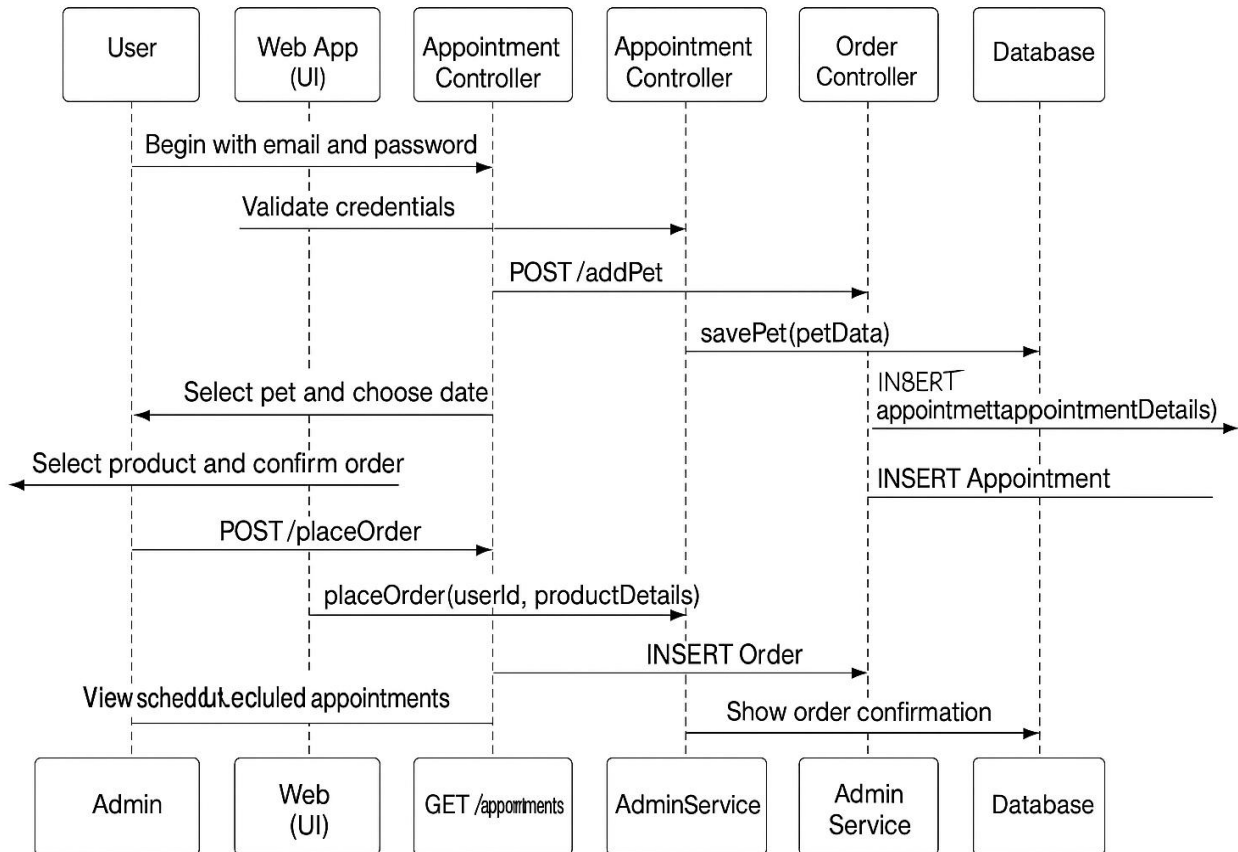


Fig 4.5: Sequence Daigram for PetHub

Following pet registration, the user can browse and select products such as pet food or accessories and proceed to book appointments or place an order. The UI helps the user to select a product and choose a preferred delivery or appointment date. Upon confirmation, the UI sends a POST /placeOrder request to the OrderController, which forwards the placeOrder(userId, productDetails) to the OrderService. The OrderService then performs an INSERT Order operation into the database, storing all relevant order information such as user ID, product ID, quantity, and total price.

After placing the order, the user receives an order confirmation. This is shown via the UI based on the response sent back by the service layer after a successful database insertion. If the order includes an appointment, such as a doctor visit or grooming session for the pet, the system proceeds to notify the admin about the scheduled appointment. The AdminService gets invoked with the `notifyAdmin(appointmentDetails)` method. This function sends a notification to the admin interface or triggers an alert to ensure the appointment gets attention and proper handling.

The user also has the option to view their scheduled appointments. When this action is performed, the Web App UI sends a request to the backend, which retrieves the user's appointment history from the database and displays it on the frontend. This allows the user to manage their pet's appointments efficiently, keep track of previous visits, and modify or cancel upcoming ones if needed.

The sequence diagram not only highlights user interactions but also the orchestration of different backend services and their communication. It showcases how microservices such as AppointmentService, OrderService, and AdminService collaborate to achieve a seamless user experience. Each service is responsible for specific tasks and maintains a clear boundary of responsibility, promoting modularity and maintainability in the system's architecture.

CHAPTER- 5

SYSTEM REQUIREMENTS

5.1 Hardware Requirements:

Processor (CPU) - Intel Core i5 or higher

RAM - Minimum 8 GB

Hard Disk - 250 GB (SSD preferred)

Display - 14 or larger monitor

Internet - Stable internet connection for deployment

5.2 Software Requirements:

Operating System - Windows 10/11, Ubuntu 20.04+, or macOS

Programming Language - Java (JDK 11 or above)

Backend Framework - Spring Boot

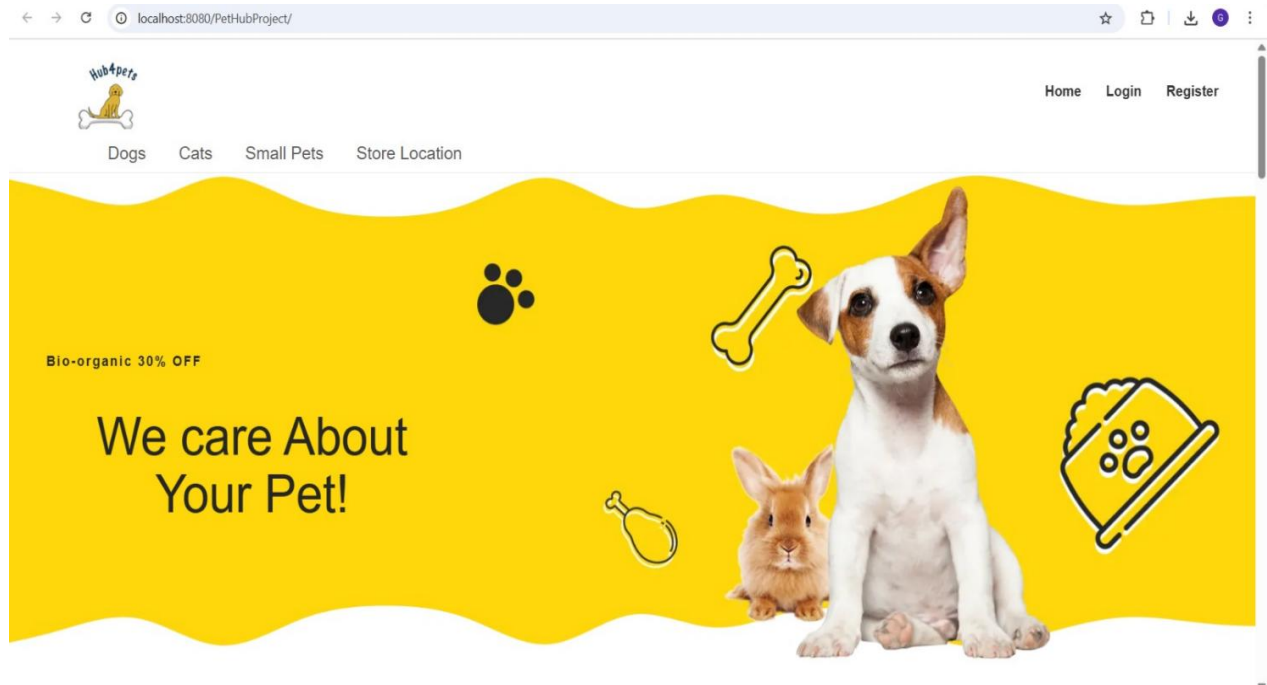
Frontend - HTML, CSS, JavaScript (with React/Angular optional)

Database - MySQL

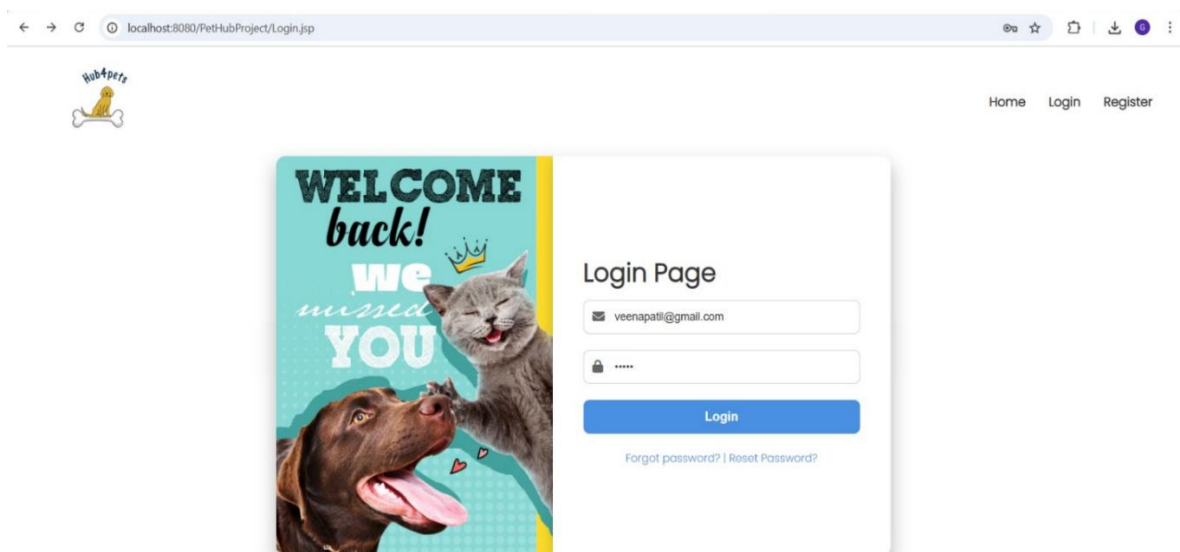
IDE - Eclipse

CHAPTER- 6

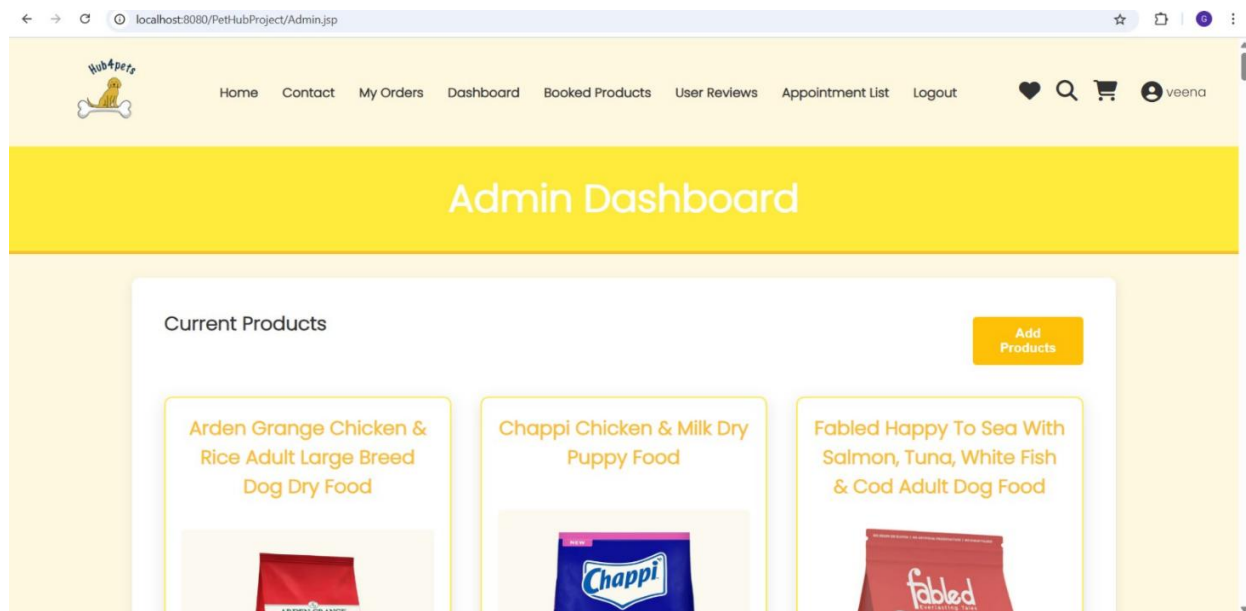
RESULTS AND DISCUSSION



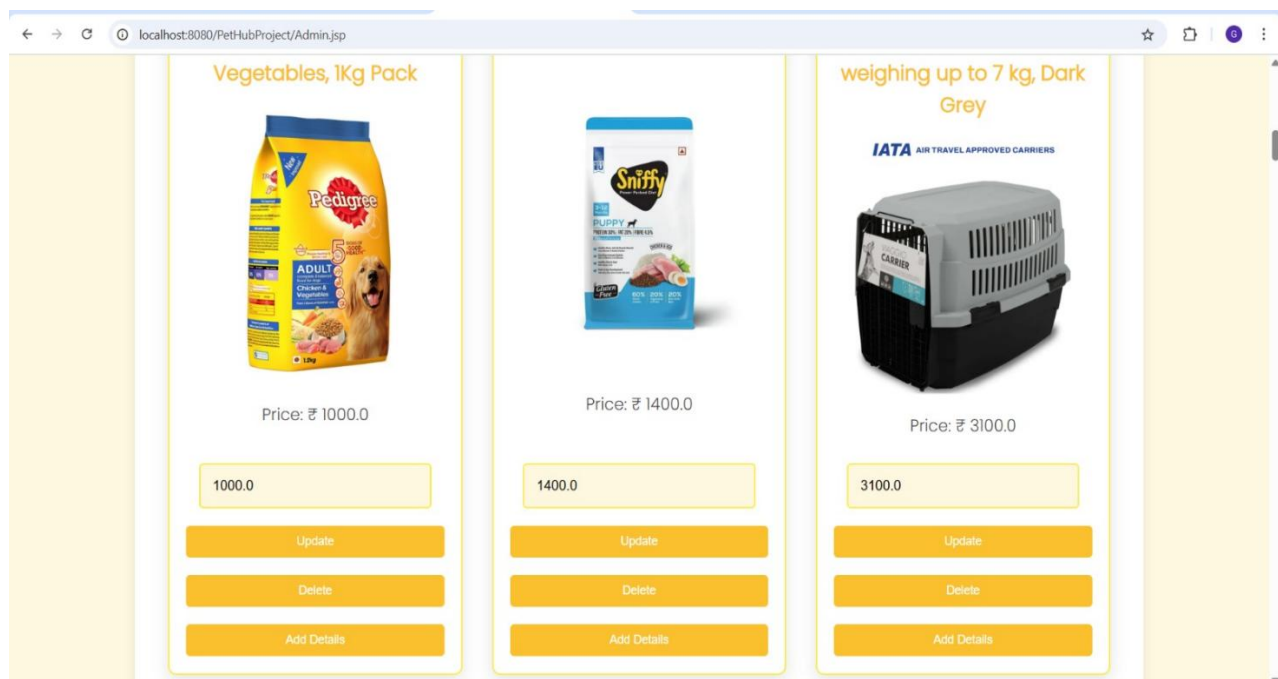
Snapshot 6.1: Admin Home Page



Snapshot 6.2: Admin Login Page

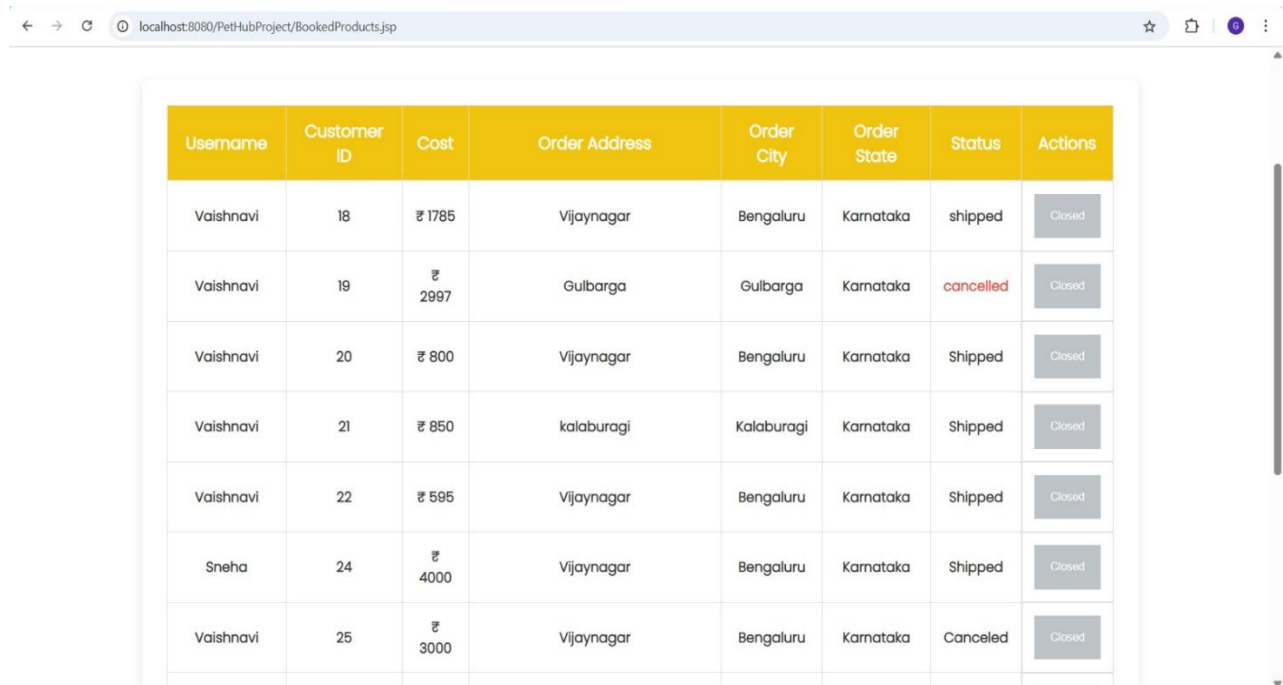


Snapshot 6.3: Admin Dashboard



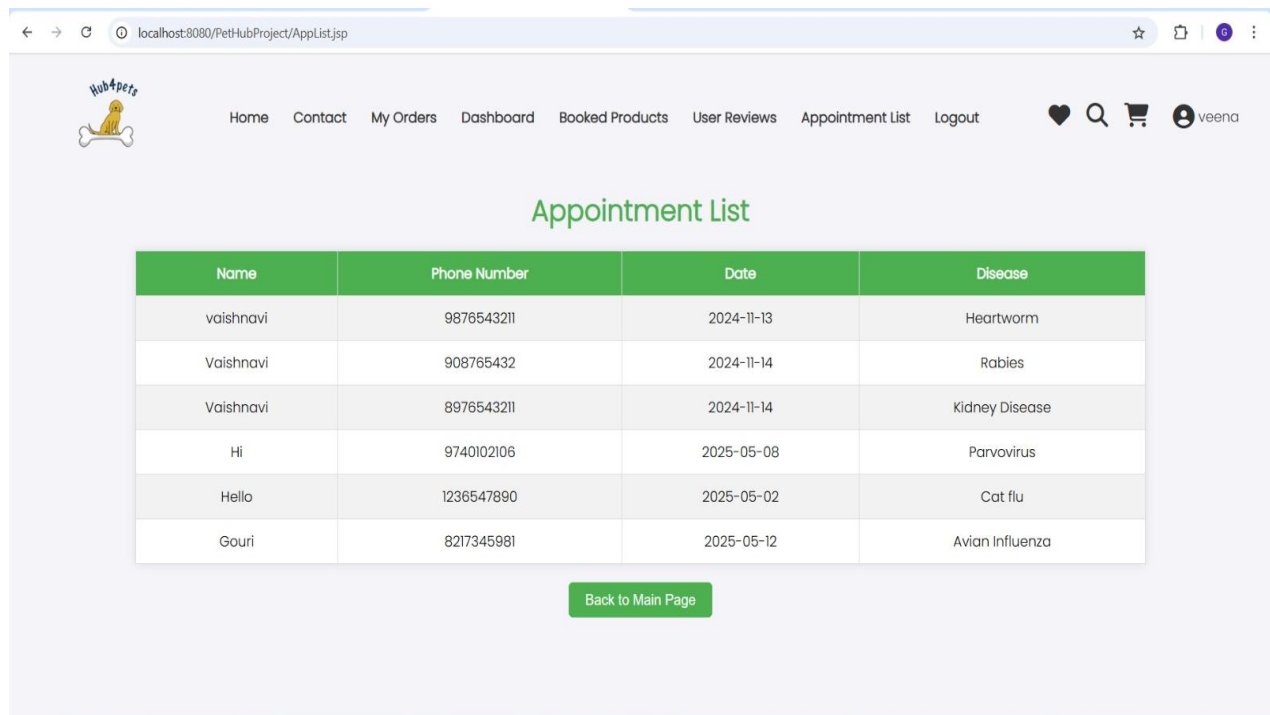
Snapshot 6.4: Product Details

PetHub



Username	Customer ID	Cost	Order Address	Order City	Order State	Status	Actions
Vaishnavi	18	₹ 1785	Vijaynagar	Bengaluru	Karnataka	shipped	<button>Closed</button>
Vaishnavi	19	₹ 2997	Gulbarga	Gulbarga	Karnataka	cancelled	<button>Closed</button>
Vaishnavi	20	₹ 800	Vijaynagar	Bengaluru	Karnataka	Shipped	<button>Closed</button>
Vaishnavi	21	₹ 850	kalaburagi	Kalaburagi	Karnataka	Shipped	<button>Closed</button>
Vaishnavi	22	₹ 595	Vijaynagar	Bengaluru	Karnataka	Shipped	<button>Closed</button>
Sneha	24	₹ 4000	Vijaynagar	Bengaluru	Karnataka	Shipped	<button>Closed</button>
Vaishnavi	25	₹ 3000	Vijaynagar	Bengaluru	Karnataka	Canceled	<button>Closed</button>

Snapshot 6.5: Ordered Products

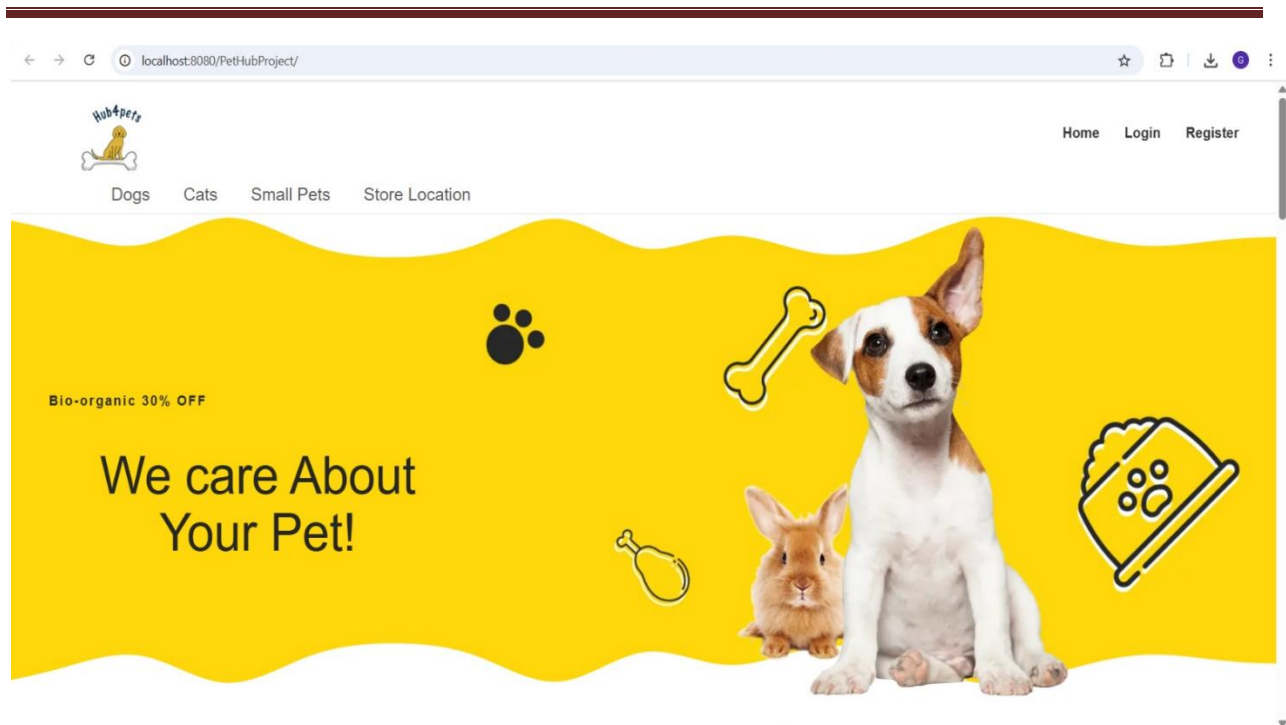


Name	Phone Number	Date	Disease
vaishnavi	9876543211	2024-11-13	Heartworm
Vaishnavi	908765432	2024-11-14	Rabies
Vaishnavi	8976543211	2024-11-14	Kidney Disease
Hi	9740102106	2025-05-08	Parvovirus
Hello	1236547890	2025-05-02	Cat flu
Gouri	8217345981	2025-05-12	Avian Influenza

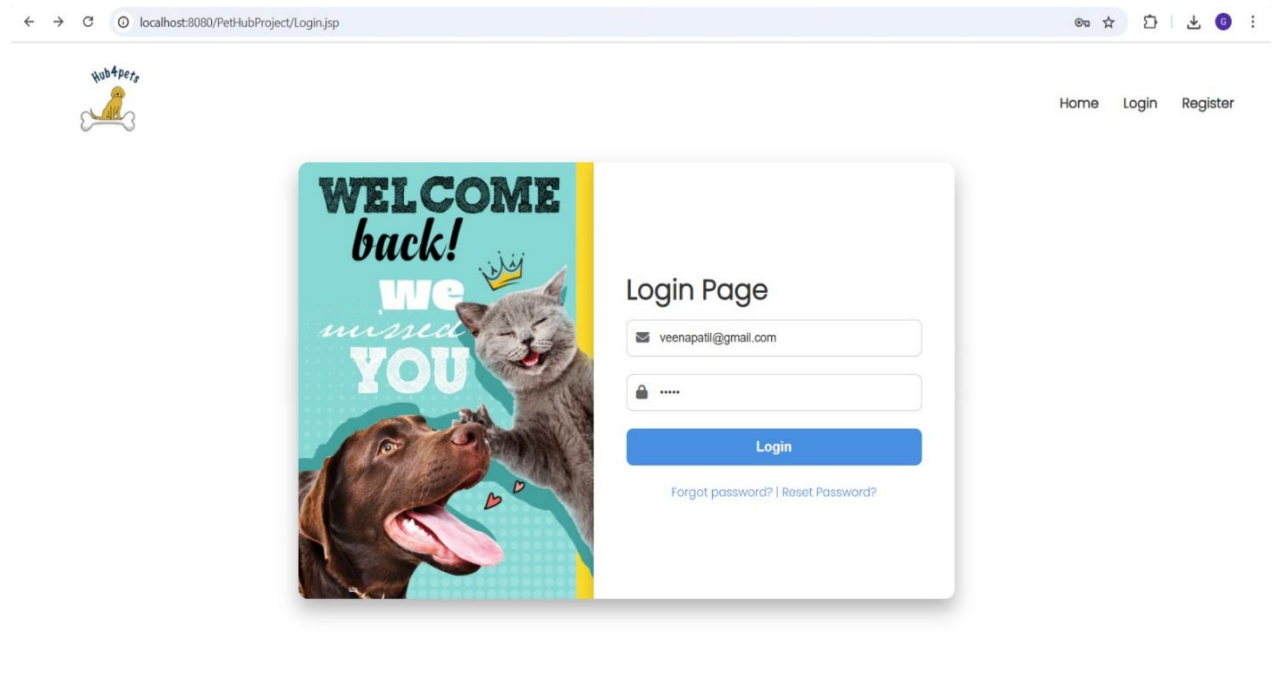
[Back to Main Page](#)

Snapshot 6.6: Appointment List

PetHub

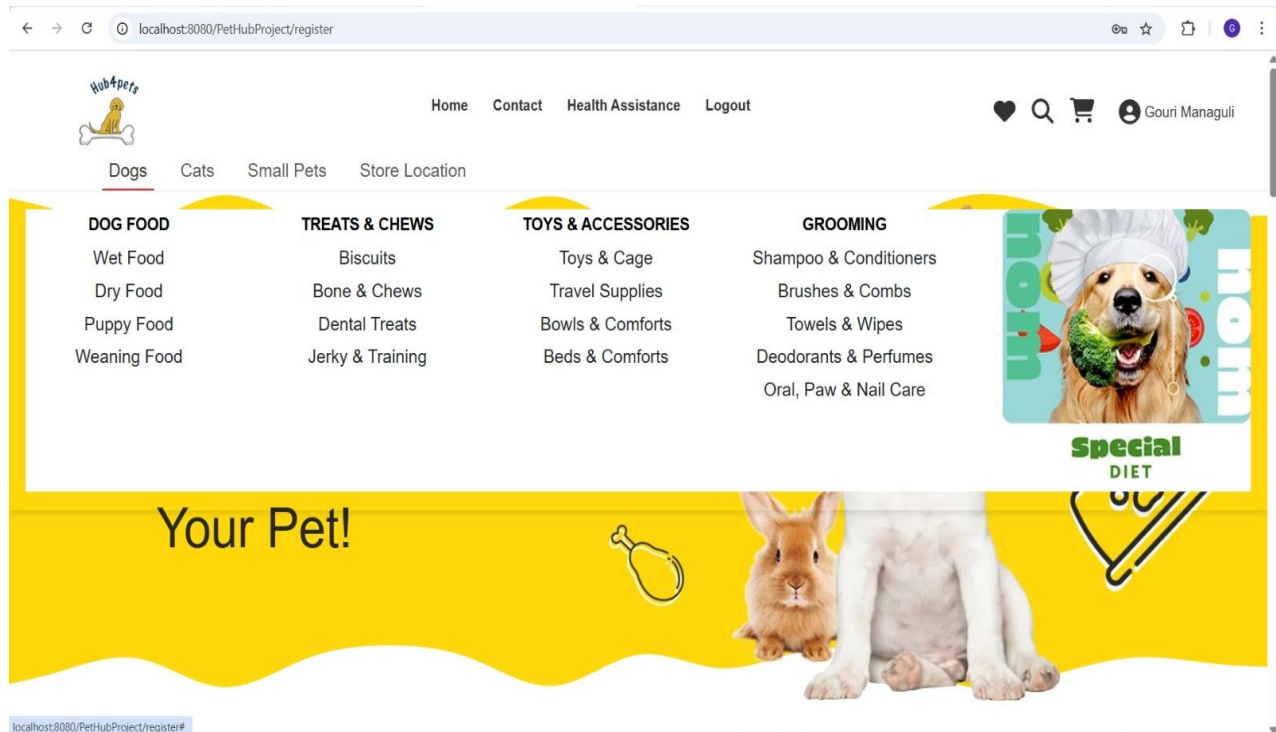


Snapshot 6.7: User Login Page

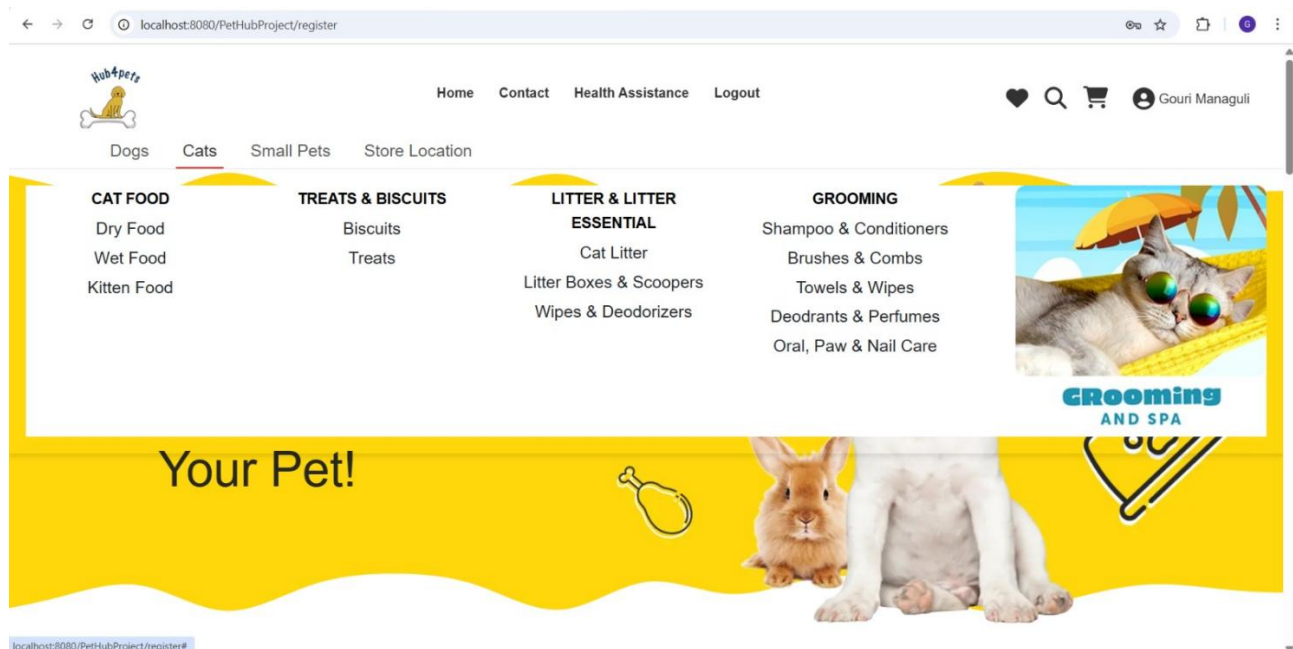


Snapshot 6.8: User Login Page

PetHub

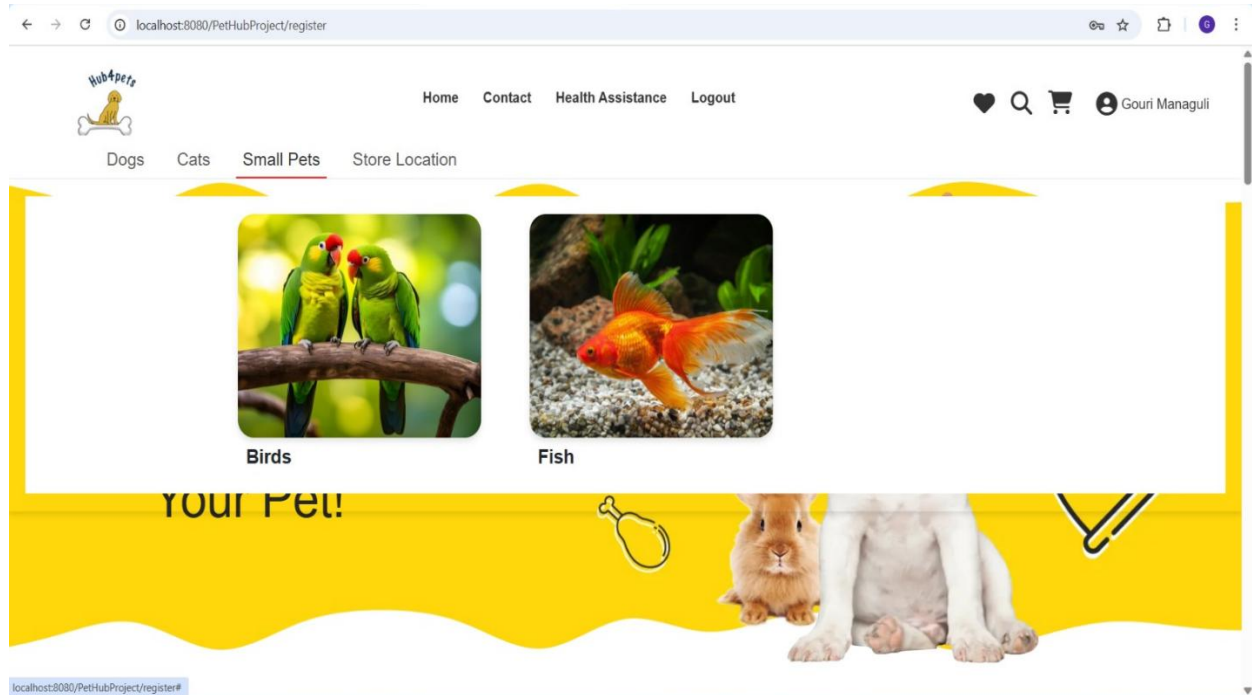


Snapshot 6.9: Accessories for Pets

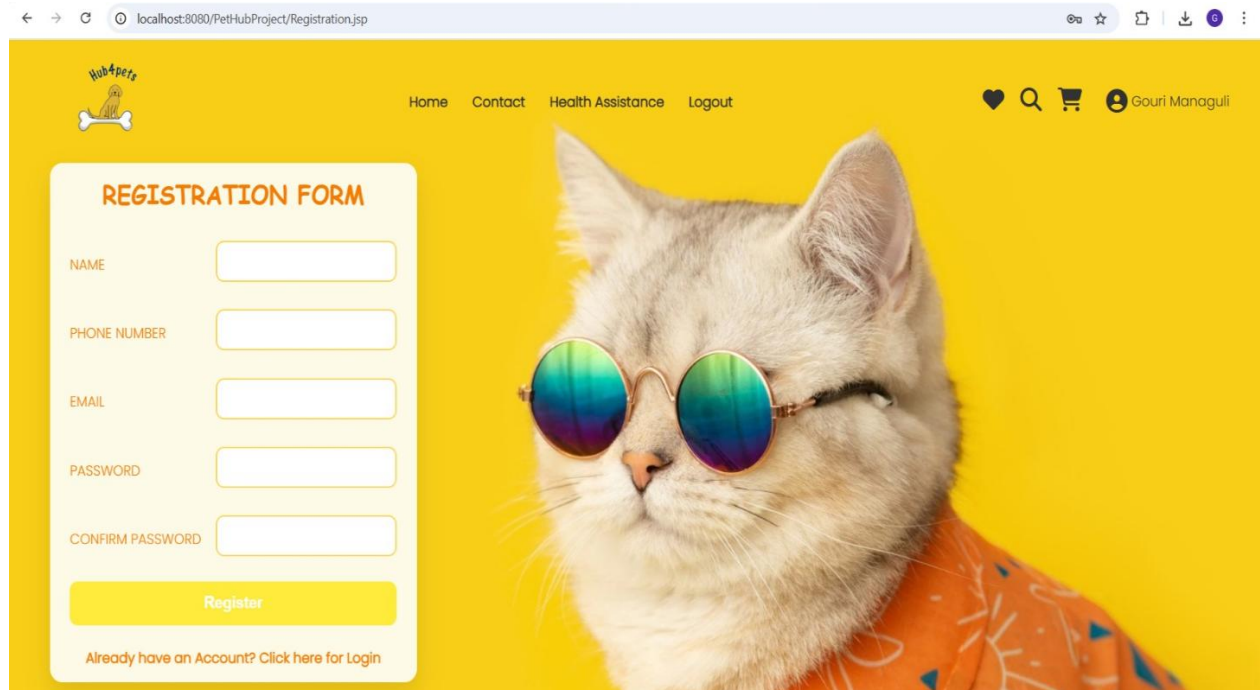


Snapshot 6.10: Accessories for Pets

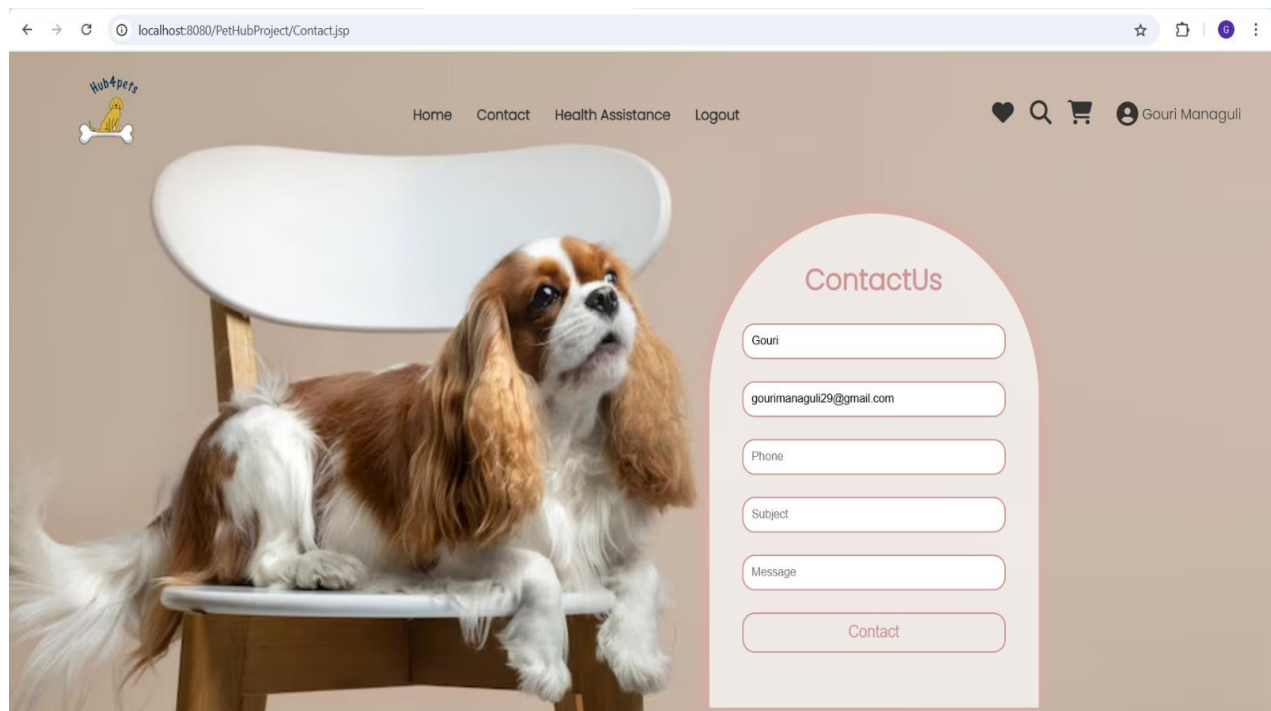
PetHub



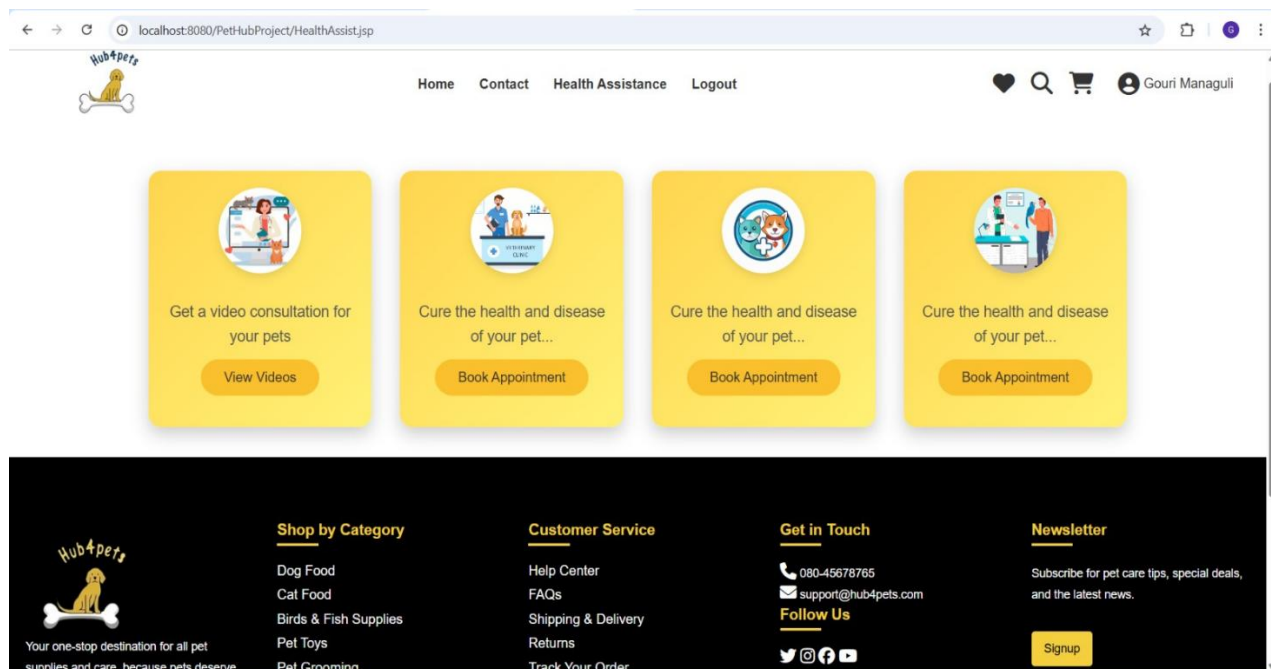
Snapshot 6.11: Small Pet Details



Snapshot 6.12: Registration Form



Snapshot 6.13: Contacting to Admin



Snapshot 6.14: Doctor Appointment for Pets

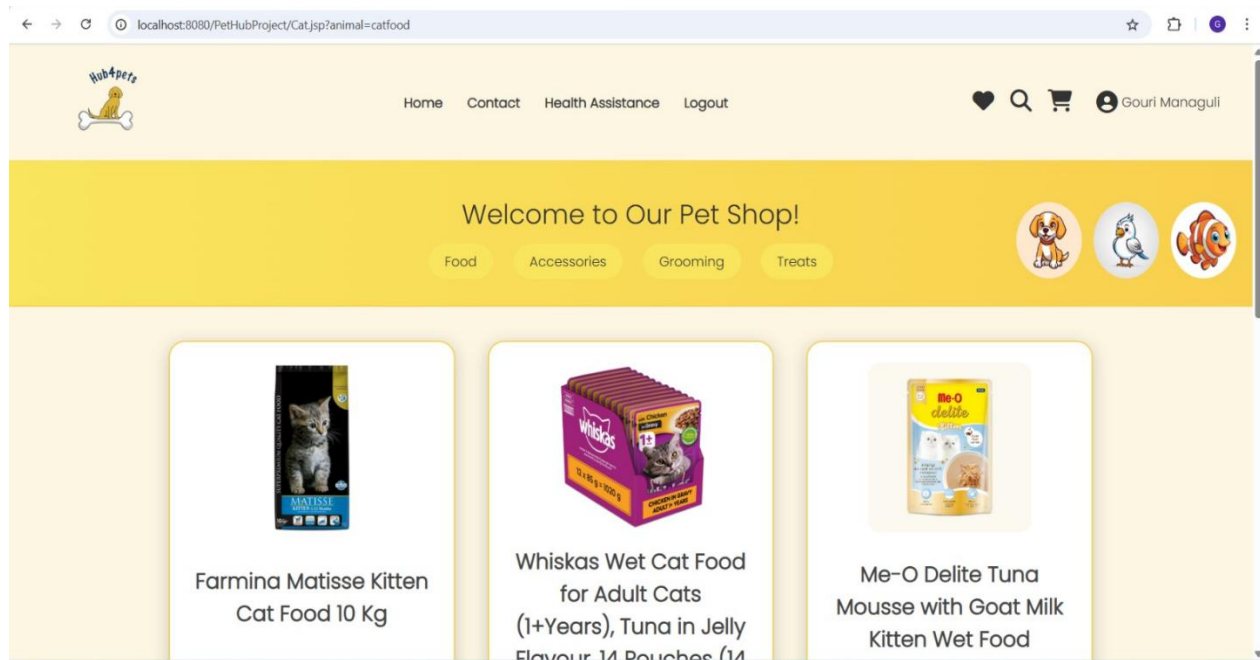
The screenshot shows a web browser window with the URL `localhost:8080/PetHubProject/BookAppointment.jsp`. The page has a blue background with a stethoscope and paw prints on the left. The navigation bar includes 'Home', 'Contact', 'Health Assistance', and 'Logout'. On the right, there are icons for a heart, search, shopping cart, and a user profile labeled 'Gouri Managuli'. The main content is a white form titled 'Book an Appointment' with the following fields: 'Name' (filled with 'Gouri'), 'Phone' (filled with '8217345981'), 'Appointment Date' (filled with '12-05-2025' and a calendar icon), and 'Select Disease' (a dropdown menu with 'Avian Influenza' selected). A yellow 'Submit' button is at the bottom of the form.

Snapshot 6.15: Booking the Appointments

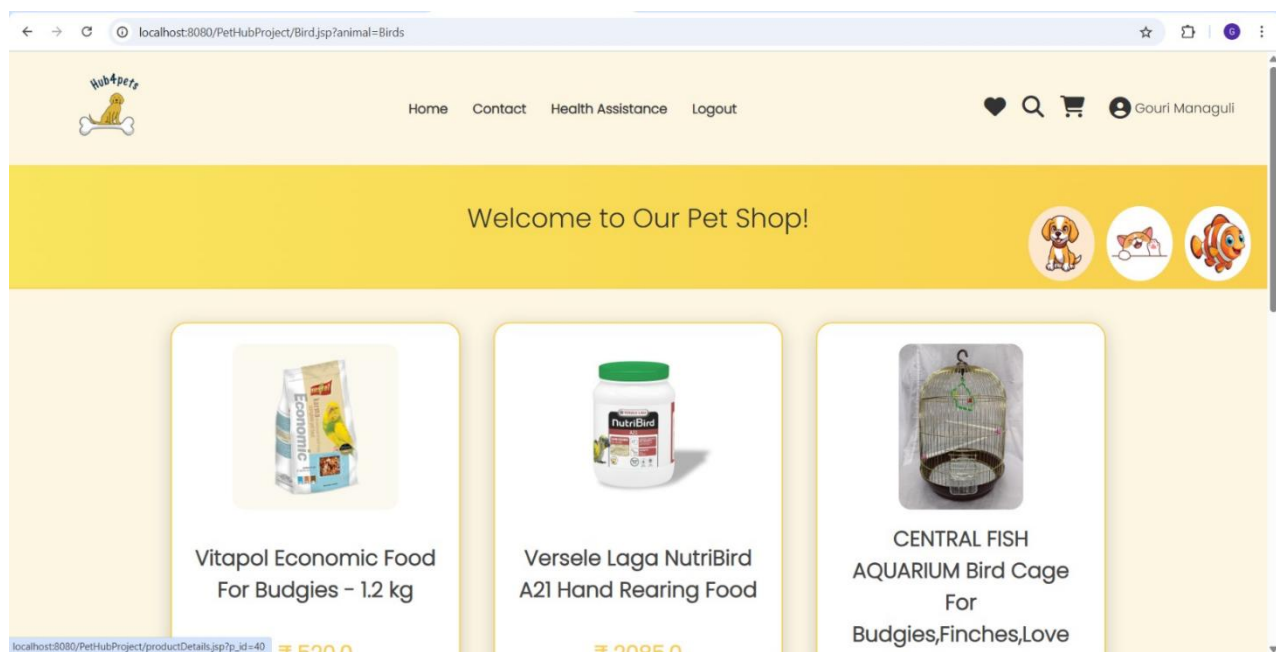
The screenshot shows a web browser window with the URL `localhost:8080/PetHubProject/Search.jsp`. The page has a white background with a yellow cloud-like shape in the center. The navigation bar is the same as in the previous snapshot. The main content is a search form with the text 'Search for animal:' above a white input field and a blue 'Search' button below it. To the right of the search area is a cartoon illustration of a golden retriever holding a bone, with a blue bowl of bones in front of it. The background features yellow wavy lines and a faint 'Free pngtree' watermark.

Snapshot 6.16: Searching for the specific Pets

PetHub

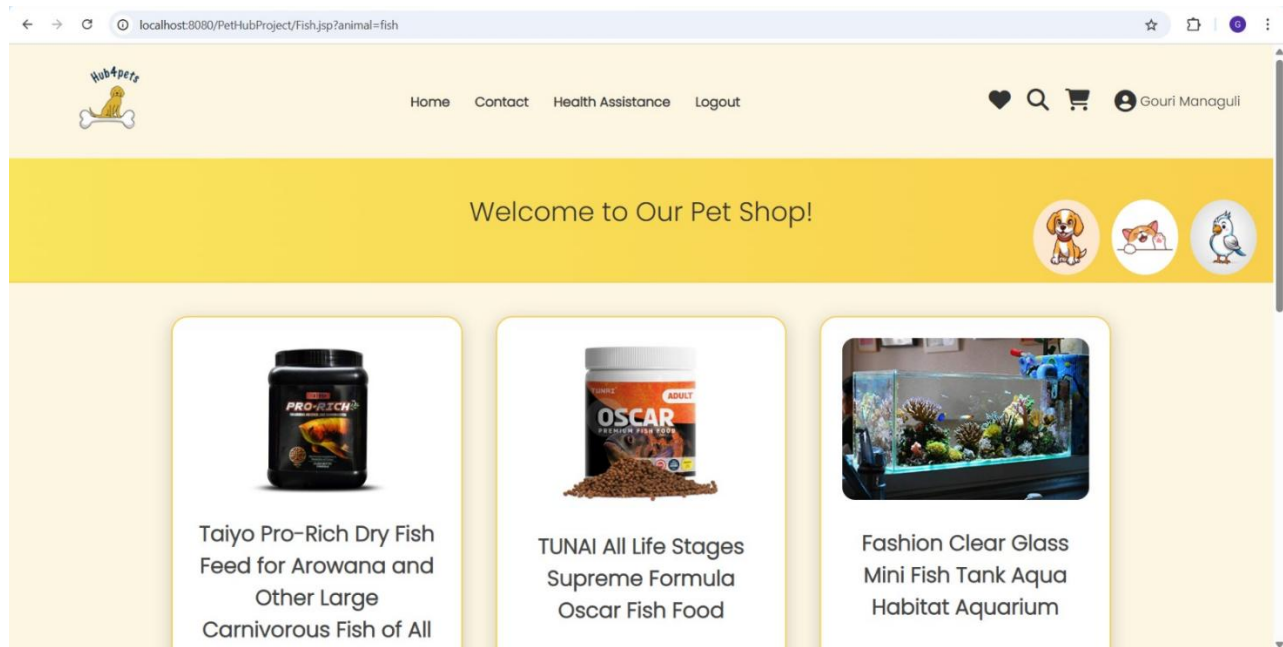


Snapshot 6.17: Product Details

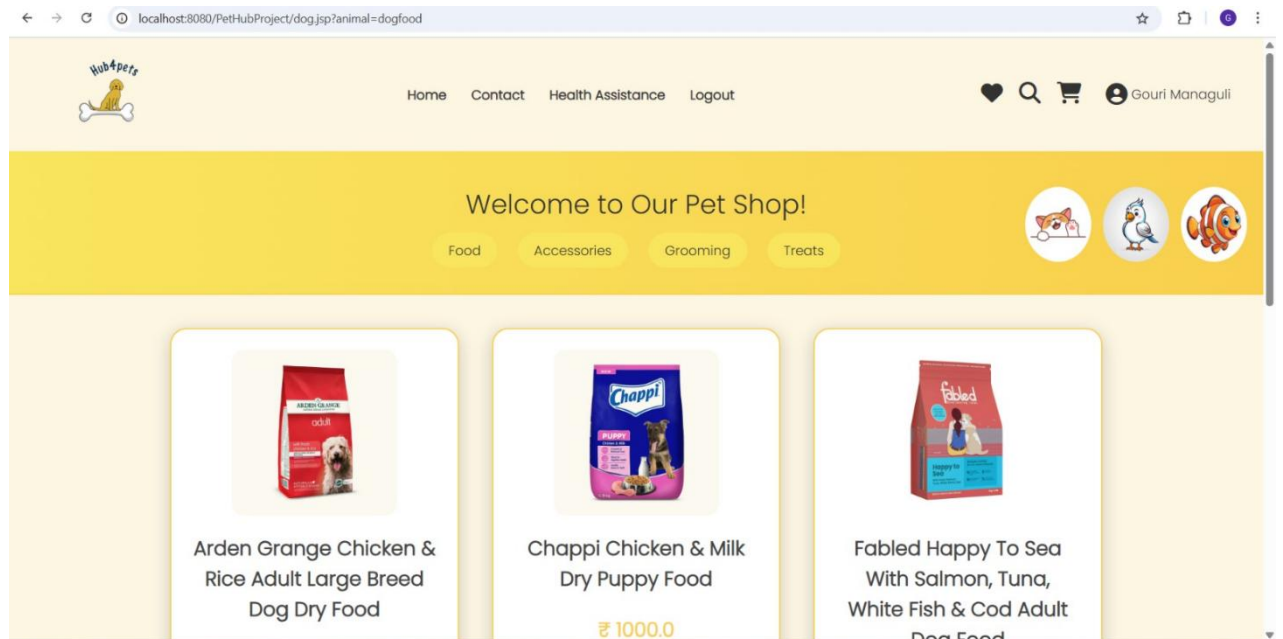


Snapshot 6.18: Product Details

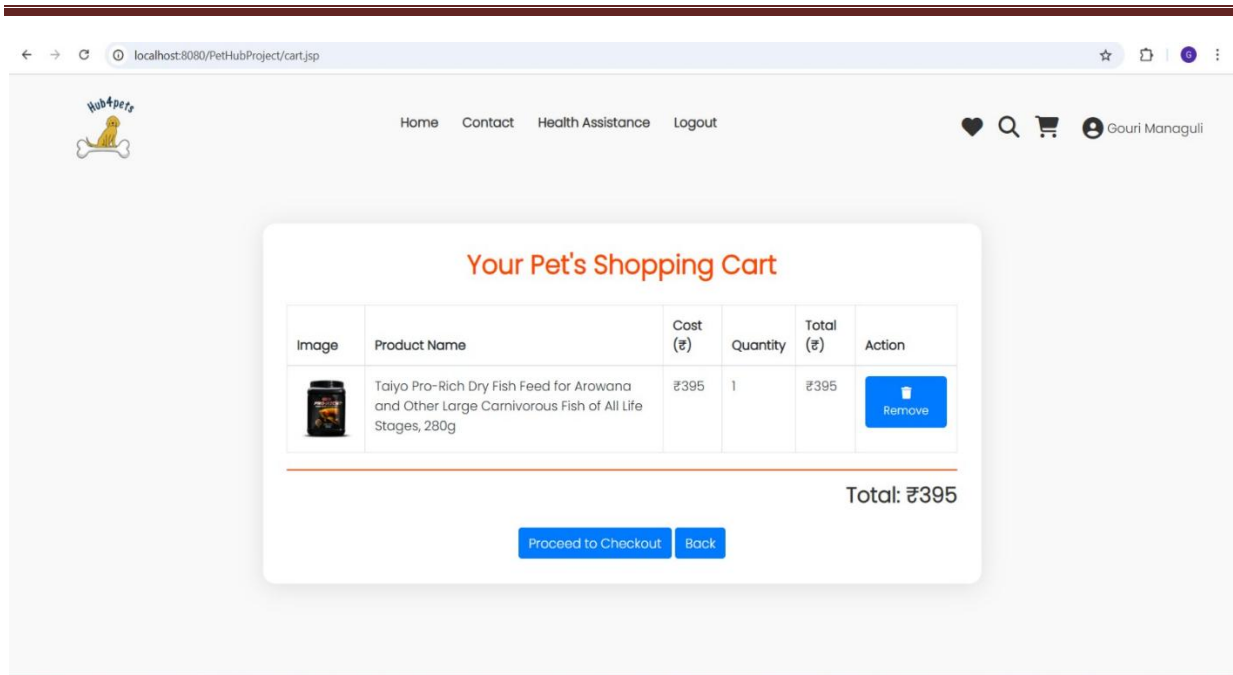
PetHub



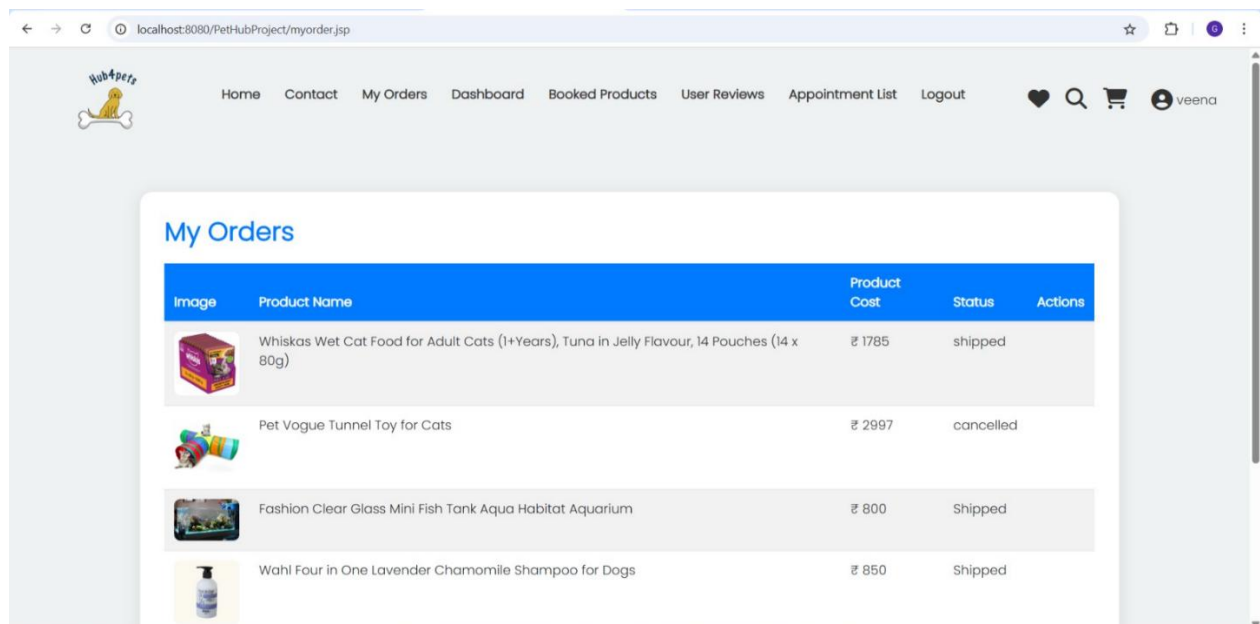
Snapshot 6.19: Product Details



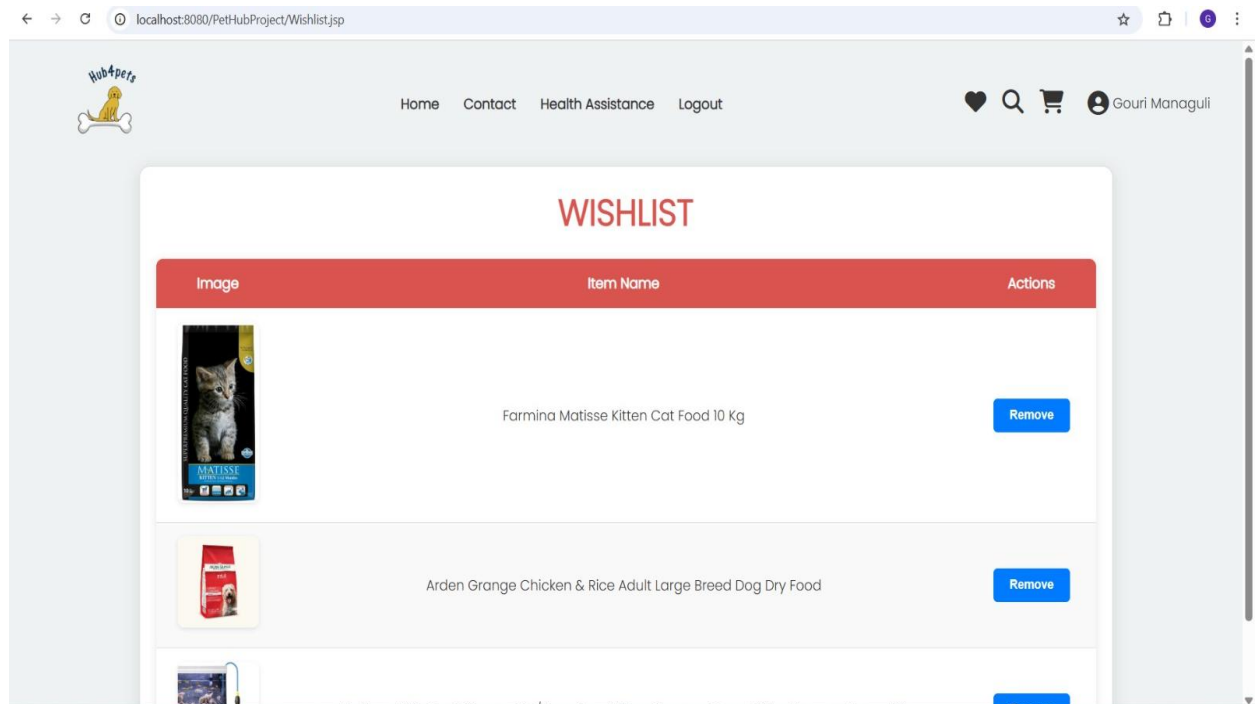
Snapshot 6.20: Product Details



Snapshot 6.21: Products added to cart



Snapshot 6.22: Ordered Products



Snapshot 6.23: Wishlist

CHAPTER- 7

ADVANTAGES AND DISADVANTAGES

7.1 ADVANTAGES

1. All-in-One Platform

- Eliminates the need for multiple apps by integrating vet services, e-commerce, and logistics into a unified interface.
- Seamlessly connects pet care needs with product purchases and health tracking.

2. Time-Saving

- Reduces hours spent on phone calls or clinic queues with one-click appointment scheduling.
- AI-powered slot recommendations optimize booking based on urgency and pet type.

3. Cost-Efficient

- Compares prices across partnered vets and stores for budget-friendly options.
- Automatic coupon applications for recurring purchases (food, meds).

4. 24/7 Accessibility

- Emergency vet chat available outside business hours for urgent queries.
- Automated order processing ensures midnight medicine deliveries.

5. Centralized Pet Profiles

- Digital vaccine cards and allergy logs shareable with any vet instantly.
- Breed-specific care plans auto-update as pets age.

6. Real-Time Tracking

- Live courier GPS maps with ETAs for temperature-sensitive items (insulin, raw food).
- Delivery rerouting via app if recipients are away.

7.2 DISADVANTAGES

1. Dependency on Third-Party Services

- Payment gateways (Stripe), map APIs, or SMS services could introduce failure points
- Cost fluctuations in external services may impact profitability

2. Data Security Risks

- Storing sensitive pet medical records increases liability for breaches
- Requires HIPAA/GDPR compliance for health data, adding regulatory overhead

3. Vet Resistance to Digitization

- Older practitioners may resist moving from paper to digital systems
- Training costs for clinics to onboard staff

4. Multi-Pet Household Complexity

- UI can become cluttered when managing 5+ pet profiles
- Confusion in assigning orders/appointments to correct pets

5. Trust Issues with Telemedicine

- Pet owners may distrust virtual consultations for serious conditions
- Liability concerns if remote diagnoses are inaccurate

6. High Customer Acquisition Cost

- Competing with established vet chains and Amazon Pet requires heavy marketing
- Price-sensitive pet owners may resist subscription fees

7. Low-Margin E-Commerce

- Pet products typically have slim profits after logistics and discounts
- Risk of becoming a "showroom" where users browse but buy elsewhere

CHAPTER- 8

INTERNSHIP OUTCOMES

During the course of my internship as a Java Full Stack Developer, I gained valuable hands on experience in designing, developing, and deploying a fully functional Event Management System. This experience significantly enhanced my understanding of both frontend and backend technologies, and deepened my practical knowledge of full stack development using

Java-based tools and frameworks:

1. Gained Practical Experience with Full Stack Architecture

I learned how to build and structure a full stack web application using the MVC (Model-View-Controller) pattern, ensuring a clean separation of concerns and maintainability.

2. Mastered Backend Development with Spring Boot

I developed secure, RESTful APIs for managing events, users, and admin operations using Spring Boot. This improved my understanding of controllers, services, repositories, and dependency injection.

3. Implemented Database Operations using MySQL

I performed CRUD operations and built database schemas using MySQL, gaining experience in writing optimized SQL queries and managing relational data.

4. Worked on Frontend Technologies

I designed responsive web pages using HTML5, CSS3, JavaScript, and Bootstrap, ensuring a user-friendly and intuitive interface for both users and administrators.

5. Integrated Frontend with Backend

I used AJAX and REST APIs to connect frontend components with the backend, enabling real-time data interaction and dynamic content rendering.

6. Enhanced Debugging and Testing Skills

I practiced debugging and validating the system across different layers, including API testing using Postman and browser-based testing of the UI.

7. Improved Version Control Practices

I used Git for source code management, learning how to commit, push, and manage branches in a collaborative environment.

8. Learned Project Deployment Techniques

I explored the basics of deploying Spring Boot applications locally and prepared the project for deployment on cloud platforms.

9. Developed Soft Skills and Team Collaboration

By working in a real-world environment, I improved my communication skills, time management, and ability to coordinate with team members.

10. Understood Real-World Application Design

I gained insight into how enterprise-level web applications are planned, developed, tested, and maintained, preparing me for future industry roles.

CONCLUSION

PetHub is an all-in-one platform that integrates veterinary services, e-commerce, and logistics for seamless pet care. It lets pet owners book appointments, manage medical records, and receive pet supplies at home. Automated reminders support timely medication and health tracking. For vets and stores, it simplifies records, inventory, and workflow management. The platform is secure, scalable, and accessible across different regions, including low-connectivity areas. IoT devices like smart collars and feeders will offer real-time health monitoring. AI will predict breed-specific health risks for early care. Telehealth features will enable virtual vet consultations, and blockchain will protect medical records.

Businesses will benefit from dynamic pricing, automated stock management, and loyalty programs. Eco-friendly deliveries will support sustainability. Community features like pet adoptions and events will boost engagement. PetHub aims to set new standards in pet care through innovation and user-focused design.

REFERENCES

1. Wang, X., & Li, Y. (2021). Design and Implementation of a Pet Management System Based on B/S Architecture. *International Journal of Computer Applications*.
2. Sharma, P., & Gupta, A. (2020). Web-Based Veterinary Appointment and Record System. *International Journal of Innovative Technology and Exploring Engineering (IJITEE)*.
3. Kumar, R., & Patel, S. (2021). Development of an Online Pet Store using PHP and MySQL. *International Journal of Advanced Computer Science and Applications (IJACSA)*.
4. Chaudhary, R., & Verma, N. (2019). Pet Health Monitoring System Using Web Technology. *International Journal of Engineering Research and Technology (IJERT)*.
5. Zhao, M., & Hu, L. (2022). Design of a Pet E-commerce Platform Using Web Front-End Technologies. *Journal of Web Engineering*.
6. Desai, K., & Shah, H. (2020). Web-Based Veterinary Clinic Management System. *International Journal of Science and Research (IJSR)*.
7. Khandelwal, R., & Jain, M. (2021). Responsive Pet Booking and Product Purchase Web Application. *International Journal of Computer Applications*.
8. Ahmed, M., & Tanveer, M. (2020). E-commerce Website for Pet Supplies Using MERN Stack. *International Journal of Advanced Research in Computer Science*.
9. Thomas, L., & Fernandes, S. (2018). Online Pet Adoption Portal using HTML5 and MySQL. *International Research Journal of Engineering and Technology (IRJET)*.